

University of Trieste
Department of Engineering and Architecture

Master's Degree in
Computer & Electronic Engineering

A Clustering–Based Strategy for Capacitated Assignment Problems

Candidate:
Lorenzo Martin DIAZ AVALOS

Supervisor:
Prof. **Lorenzo CASTELLI**

Co–Supervisor:
Dott. **Andrea GASPARIN**

Academic Year 2024/2025

"Sometimes it's the very people who no one imagines anything of who do the things that no one can imagine"

— **The Imitation Game**

Abstract

Large-scale assignment problems, where numerous entities compete for a limited set of shared resources, are ubiquitous in domains ranging from logistics to transportation networks. Mixed-Integer Linear Programming (MILP) provides a rigorous and flexible framework for modeling these scenarios, allowing for the precise definition of constraints and objectives. However, global MILP formulations often become computationally intractable as instance size grows, due to the rapid expansion of the model and the strong structural coupling induced by capacity constraints. In this thesis, we propose a scalable clustering-based decomposition strategy to address this scalability barrier. We develop a pipeline that (i) partitions entities into size-constrained clusters using a Neighbor Joining algorithm based on interaction similarity, (ii) solves each cluster as an independent subproblem, and (iii) restores global feasibility through a targeted repair mechanism that re-optimizes only the resources involved in cross-cluster violations. We experimentally evaluated our proposal on synthetic grid-based instances of two distinct problem families: Space-Time Capacitated Path Assignment and Capacitated Facility Location (both single-source and splittable-demand). The results demonstrate that the approach achieves orders-of-magnitude speedups with negligible optimality gaps for facility location problems and remains effective for path assignment when interactions are moderate, providing a robust method to exploit local structures in large-scale combinatorial optimization.

Sommario

I problemi di assegnamento su larga scala, in cui numerose entità competono per un insieme limitato di risorse condivise, sono onnipresenti in domini che spaziano dalla logistica alle reti di trasporto. La programmazione lineare intera mista (MILP) fornisce un quadro rigoroso e flessibile per modellare questi scenari, consentendo la definizione precisa di vincoli e obiettivi. Tuttavia, le formulazioni MILP globali diventano spesso computazionalmente intrattabili al crescere della dimensione dell'istanza, a causa della rapida espansione del modello e del forte accoppiamento strutturale indotto dai vincoli di capacità. In questa tesi, proponiamo una strategia di decomposizione scalabile basata sul clustering per superare questa barriera di scalabilità. Sviluppiamo una pipeline che (i) partiziona le entità in cluster di dimensione limitata utilizzando un algoritmo di Neighbor Joining basato sulla similarità di interazione, (ii) risolve ogni cluster come un sottoproblema indipendente e (iii) ripristina la fattibilità globale attraverso un meccanismo di riparazione mirato che ri-ottimizza solo le risorse coinvolte nelle violazioni tra cluster. Valutiamo sperimentalmente la nostra proposta su istanze sintetiche su griglia di due distinte famiglie di problemi: lo Space-Time Capacitated Path Assignment e il Capacitated Facility Location (sia single-source che splittable-demand). I risultati dimostrano che l'approccio ottiene speedup di ordini di grandezza con gap di ottimalità trascurabili per i problemi di facility location e rimane efficace per l'assegnamento di path quando le interazioni sono moderate, fornendo un metodo robusto per sfruttare le strutture locali nell'ottimizzazione combinatoria su larga scala.

Contents

Abstract I
Sommario III

1 Introduction **1**
1.1 Motivations and Contributions 2
1.2 Case Studies and Evaluation Overview 2
1.3 Dissertation Outline 3

2 Background **5**
2.1 Scalability Challenges in Mixed Integer Linear Programming . . 5
2.2 Algebraic Decompositions 8
2.3 Clustering Decomposition and Matheuristic Approaches 10

3 Methodology **13**
3.1 Problem Scope and Applicability Conditions 13
3.2 Proposed Strategy 15
3.2.1 Abstract Components 15
3.2.2 Main Pipeline 17
3.2.3 Feasibility restoration 17
3.3 Presolve Indicators 19
3.3.1 Similarity Indicators 19
3.3.2 Congestion Indicators 22

4 Similarity Modeling and Neighbor Joining Clustering **25**
4.1 Similarity Function Design 25
4.1.1 Design Principles 26
4.1.2 A General Footprint Template 26
4.1.3 Similarity Families 27
4.1.4 From Similarities to Distances 28
4.2 Neighbor Joining for Constrained Hierarchical Clustering 28
4.2.1 The Algorithm 29
4.2.2 Computational Aspects 30
4.2.3 Extracting a Partition 31

5 Mathematical Models **33**
5.1 A General Mathematical Framework 33
5.2 The Space–Time Capacitated Path Assignment Problem 37
5.3 The Capacitated Facility Location Problem 40
5.3.1 Single–Source 41
5.3.2 Splittable–Demand 42

6 Experimental Settings **45**
6.1 Instance Generation 45
6.2 Parameters Selection 47
6.3 Evaluation Protocol 49

7 Results **53**
7.1 RQ1: Analysis of the Solution Quality 53
7.2 RQ2: Analysis of the Execution Time 56
7.3 RQ3: Computational Advantage 58
7.4 RQ4: Internal Dynamics 60
7.5 RQ5: Analysis of Presolve Indicators 63

8 Conclusions and Future Work **67**
8.1 Conclusions 67
8.2 Future Work 68

A Detailed Experimental Results **71**
A.1 Space-Time Capacitated Path Assignment 72
A.2 Single Source Capacitated Facility Location 73
A.3 Splittable-Demand Capacitated Facility Location 74
A.4 Correlation Analysis of Presolve Indicators 75

Chapter 1

Introduction

Many large-scale optimization problems share a common pattern: a set of decision entities must take discrete (or partially discrete) choices while competing for a limited pool of shared resources. Typical examples include demand classes competing for space–time capacities in transportation networks, or clients competing for facility capacities in location–allocation systems. Mixed Integer Linear Programming provides a flexible modeling framework for these settings, but global formulations can become computationally prohibitive as the instance size grows. The difficulty is not only due to integrality and combinatorial explosion, but also to the structural coupling induced by capacity constraints: as the number of entities increases and their set of admissible choices becomes richer, the number of variables and constraints expands rapidly and decisions become intertwined across the entire instance. A key observation motivating this thesis is that in resource-sharing problems, not all entities interact equally. Some entities are strongly coupled because they tend to load the same scarce resources, whereas others remain almost independent. If we can approximate this interaction structure directly from the instance data and from the available choice sets, we can exploit it to control model growth: strongly interacting entities should be optimized together, while weakly interacting entities can be separated to reduce coupling at a limited cost in solution quality. Building on this idea, the thesis develops a scalable *clustering–based strategy* for capacitated assignment problems. The approach groups entities into bounded-size clusters, solves a sequence of smaller MILP subproblems, and combines their solutions into a global draft. Because the global coupling constraints are relaxed across clusters during the independent subproblem solves, the merged draft may violate shared capacities; when this happens, feasibility is restored through a targeted re-optimization step focused on the most critical resources. This design yields a controllable computational envelope: cluster models are bounded by construction, and global coordination is limited to a reduced residual optimization rather than a full global re-solve.

1.1 Motivations and Contributions

This work is motivated by the need to scale beyond the limits of global MILP resolution while keeping the methodology modular and easy to instantiate. Classical exact decomposition paradigms (*e.g.*, column generation, Benders-type methods, Lagrangian relaxations) can exploit specific algebraic structures, but they often require substantial reformulation effort and specialized algorithmic machinery. In contrast, the approach developed in this thesis leverages an entity-level view of interaction, so that strongly coupled entities are handled together, while weakly coupled ones can be separated with controlled loss in solution quality.

Within this perspective, the thesis contributes (i) an interaction-driven partitioning mechanism based on pairwise similarity information derived from the problem data and from option-induced resource footprints; (ii) a size-constrained hierarchical clustering backbone built via *Neighbor Joining* (NJ) and cut to enforce an explicit maximum cluster size, turning this parameter into a direct and interpretable control knob; (iii) a targeted feasibility-restoration step that resolves cross-cluster capacity conflicts through reduced global re-optimization around critical bottlenecks; and (iv) an extensive computational study that characterizes how solution quality and runtime evolve as both instance size and interaction intensity increase, quantifying the speedups obtained relative to a global MILP baseline and identifying regimes in which the reduced global re-optimization becomes the dominant cost driver.

1.2 Case Studies and Evaluation Overview

The framework is validated on two problem families that share the same structural backbone: many entities making choices under shared capacity constraints, but differing in modeling assumptions and in how feasibility restoration operates.

Space–Time Capacitated Path Assignment. The first case study is the *space–time capacitated path assignment* problem. Entities are *Origin–Destination* (OD) pairs, each associated with an integer demand represented as a number of indivisible travel units. The underlying network is time-expanded, and capacities are enforced on space–time resources, so congestion is driven by temporal overlap of traversals. For each OD pair, a finite candidate set of admissible paths is pre-generated, and the optimization allocates the demand across these candidates while respecting capacities. In this setting, feasibility restoration naturally operates at the level of indivisible travel units, enabling fine-grained adjustments around overloaded space–time bottlenecks.

Capacitated Facility Location. The second case study is the *capacitated facility location* problem, considered in two variants: *single-source*, where each client must be assigned to exactly one open facility, and *splittable-demand*, where

a client's demand may be distributed across multiple open facilities. Here, shared resources are facility capacities, and the model includes facility opening decisions. The two variants induce different restoration dynamics after merging: SS-CFL requires discrete reassignment of whole clients, whereas SD-CFL can shift fractional flows, typically enabling smoother feasibility recovery.

Evaluation protocol. The computational study is conducted on synthetic grid-based instances designed to scale both *the size* and *intensity of the interaction*. Instance size is increased by populating multiple spatial regions (quadrants) while preserving local density, so that the number of entities grows approximately linearly with the number of populated quadrants. Interaction across regions is controlled by an offset parameter that ranges from strict separation (weak cross region coupling) to partial overlap (strong coupling). For each configuration, we compare the proposed strategy with a global MILP baseline under a fixed time limit, using multiple random seeds to account for structural variability. Performance is assessed along two axes: (i) solution quality, measured through the relative gap with respect to the best global baseline solution available under the same computational limits, and (ii) computational effort, reported both as total runtime and as an internal breakdown highlighting the cost of clustering versus the cost of reduced global re-optimization.

1.3 Dissertation Outline

The remainder of the thesis is organized as follows. Chapter 2 reviews the challenges of scalability in MILP and summarizes the decomposition and matheuristic ideas relevant to large-scale resource-sharing problems. Chapter 3 introduces the proposed clustering strategy at the abstract level and discusses the conditions under which it is applicable. Chapter 4 details the similarity modeling choices and the NJ-based hierarchical clustering mechanism used to obtain size-feasible partitions. Chapter 5 presents the mathematical models for the case studies considered and explains how they instantiate the general pipeline, including the feasibility restoration step. Chapter 6 describes the experimental settings, including instance generation, parameter selection, and evaluation metrics. Chapter 7 reports and discusses the computational results. Chapter 8 concludes the thesis and outlines the direction for future work. Detailed numerical results are provided in the Appendix A.

Chapter 2

Background

Large-scale optimization problems involving shared resources arise in a wide range of applications, including transportation networks, facility location, and logistics planning. Although Mixed-Integer Linear Programming (MILP) provides a unified and expressive modeling framework for such problems, its direct application often becomes computationally prohibitive as the size of the instance and the level of detail increase. The difficulty is not merely combinatorial in nature, but structural: realistic formulations typically involve a large number of decision variables, strong global coupling through capacity constraints, and weak continuous relaxations. This chapter provides the methodological background necessary to motivate the decomposition strategy proposed in this thesis. We first analyze the fundamental limitations of the scalability of global MILP formulations, with particular emphasis on resource–sharing problems that exhibit block-angular structure. We then review classical algebraic decomposition techniques and discuss their strengths and limitations when applied to large heterogeneous instances. Finally, we introduce clustering-based and matheuristic approaches as practical alternatives that trade exact optimality guaranties for controlled model size and execution time. The purpose of this chapter is not to exhaustively survey all solution techniques for large-scale MILP, but to establish a clear diagnostic understanding of why global formulations fail to scale, which structural properties drive this failure, and how clustering–based decomposition can be viewed as a principled response to these limitations. This perspective directly informs the design of the clustering-based strategy developed in the remainder of the thesis.

2.1 Scalability Challenges in Mixed Integer Linear Programming

MILP is a cornerstone of discrete optimization, offering a flexible and mathematically rigorous framework for modeling decision problems with a combinatorial structure. Despite decades of algorithmic progress, solving large-scale MILPs

to proven optimality remains computationally challenging. From a complexity-theoretic point of view, the MILP decision versions are $\mathcal{NP}$–complete in general [1], and even restricted problem classes can exhibit exponential worst–case behavior under exact solution methods such as branch–and–bound or branch–and–cut [2]. Modern commercial solvers, most notably CPLEX, Gurobi, and SCIP, have significantly advanced the state of the art through presolve reductions, cutting plane separation, primal heuristics, and sophisticated branching strategies [3, 4]. Nevertheless, empirical evidence from benchmark libraries such as MIPLIB 2017 [5] shows that a substantial fraction of large and structurally complex instances remain intractable within reasonable memory or time limits. In practice, the scalability barrier is driven not by a single factor but by the interaction of the dimensionality of the model, the global structural coupling, and weak continuous relaxations.

Dimensionality and Model Granularity.

A first and immediate obstacle to scalability is the rapid growth of model dimensions as problem size and modeling details increase. In many resource–sharing and network optimization problems, the number of decision variables does not scale linearly with the number of entities, but multiplicatively or combinatorially. Time–expanded network formulations, for example, replicate spatial decision variables across discrete time steps, generating a number of variables and constraints proportional to the time horizon [6]. While time expansion enables a precise representation of temporal feasibility and congestion, it often leads to pseudo–polynomial growth in model size and severe memory pressure when fine time discretizations are required. A similar phenomenon arises in path–based formulations of routing and multi–commodity flow problems. The number of feasible paths between an origin–destination pair can grow exponentially with the size of the underlying graph. To maintain tractability, practitioners typically restrict attention to a finite set of candidates such as k shortest paths, randomized shortest paths, or penalty–based rerouting schemes [7]. This restriction introduces a structural trade–off: larger candidate sets improve solution quality and robustness, but substantially increase the number of binary or integer variables and the cost of solving the associated LP relaxations. As model granularity increases to capture operational realism through atomic demand units, discrete assignments, or fine-grained capacity tracking, the constraint matrix becomes high–dimensional and less amenable to aggressive presolve. Even when sparsity is preserved, the repeated solution of large LP relaxations across the branch–and–cut tree becomes a dominant computational bottleneck.

Global Coupling and Block–Angular Structure.

Beyond dimensionality, the complexity of many large-scale MILPs is governed by the topology of their constraint matrix. A broad class of resource-sharing problems exhibits a *primal block–angular structure* [8]. Let the decision variables be partitioned into K disjoint subvectors $x_1, \ldots, x_K$, representing distinct

entities. The problem minimizes a separable objective function subject to independent local constraints $D_k x_k \leq d_k$ and a set of global coupling constraints $\sum_{k=1}^{K} A_k x_k \leq b_0$. Consequently, the full constraint matrix $\mathcal{A}$ adopts a characteristic sparse block-diagonal shape with a dense border:

$$\mathcal{A} = \begin{pmatrix} A_1 & A_2 & \dots & A_K \\ D_1 & 0 & \dots & 0 \\ 0 & D_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & D_K \end{pmatrix} \tag{2.1}$$

Here, the diagonal blocks D_k correspond to easy-to-solve subproblems, while the submatrices A_k (linking constraints) couple the variables [9]. This structure poses specific challenges for global solvers. First, the linking rows A_k connect otherwise disjoint columns, causing catastrophic fill–in during basis factorization and destroying the sparsity advantage [10]. Second, the coupling constraints typically provide a weak LP relaxation: conflicts that are strictly infeasible at the integer level (*e.g.*, resource contention) are often satisfied fractionally in the relaxation, delaying their detection until deep in the branch-and-bound tree [11]. General–purpose presolve and cut generation routines are typically unaware of the high–level interaction structure among entities unless this structure is explicitly encoded through reformulation or decomposition. As a result, global formulations often fail to capitalize on the latent near–separability present in large instances.

Integrality Gap and Search Tree Explosion.

The third critical driver of computational difficulty is the quality of the LP relaxation. The efficiency of branch–and–cut algorithms depends on the availability of strong bounds that allow early pruning of suboptimal nodes. However, in many capacitated allocation and network design problems, the integrality gap, defined as the relative difference between the optimal integer solution and the optimal LP–relaxation, can be substantial [12]. In facility location, network design, and related problems, natural formulations often rely on Big–M constraints or weak link inequalities, allowing fractional solutions that significantly underestimate the true total cost [13]. Although modern solvers implement families of strengthening cuts, such as flow–cover, knapsack–cover, and mixed–integer rounding cuts, their separation can be computationally expensive, and the number of required cuts may grow rapidly with instance size. A large integrality gap leads directly to a search tree explosion: many nodes must be explored before the lower bound approaches the incumbent solution value. Empirically, it is often observed that feasible high-quality solutions can be found relatively quickly using heuristics, while proving optimality requires orders of magnitude more computational effort [14, 15]. This asymmetry motivates approaches that prioritize scalability and feasibility over exact optimality guaranties.

Implications for Decomposition and Clustering–Based Approaches.

Together, these phenomena explain why simply increasing computational power is insufficient to overcome the scalability barrier of global MILP formulations [11, 16]. Dimensionality growth, global coupling, and weak relaxations reinforce each other, leading to excessive memory consumption and prohibitive runtimes [17]. This observation motivates decomposition strategies that explicitly control the model size and manage global coupling. Classical algebraic decomposition methods, such as Dantzig–Wolfe decomposition, Benders decomposition, and Lagrangian relaxation, exploit block–angular structure to restore separability at the algorithmic level. However, their practical effectiveness often hinges on nontrivial reformulations, specialized branching rules, and careful algorithmic tuning. An alternative more flexible response is to adopt clustering–based matheuristics, which decomposes the problem at the level of entities rather than constraints. By grouping strongly interacting entities and solving subproblems of a bounded size, such approaches limit model growth by design and postpone the handling of residual coupling to a controlled repair or coordination phase.

2.2 Algebraic Decompositions

Many large-scale mixed–integer linear programs arising in practice exhibit an inherent block–angular structure, in which independent or weakly interacting subsystems are coupled through a relatively small set of global constraints. Algebraic decomposition methods exploit this structure by reformulating the original problem to restore separability at the algorithmic level. Classical approaches in this class include Dantzig–Wolfe decomposition, Benders decomposition, and Lagrangian relaxation, each of which addresses global coupling through a different mathematical mechanism.

Dantzig–Wolfe Decomposition.

The Dantzig–Wolfe decomposition was originally developed for large linear programs with block–angular constraint matrices [9]. The key idea is to reformulate the original problem by replacing each block of variables with its convex hull representation. This yields a master problem that coordinates the blocks through linking constraints and a collection of independent subproblems that generate extreme points or rays via column generation. When extended to mixed–integer programs, the Dantzig–Wolfe decomposition provides a tight linear relaxation corresponding to the convex hull of each block, often significantly strengthening the bound compared to the global LP relaxation [18]. However, its practical application to MILP faces several challenges. First, the number of columns required to approximate the convex hull may be exponential, making column generation computationally demanding. Second, the integration of branching decisions with column generation requires specialized branch-and-price schemes,

which are substantially more complex to implement and tune than standard branch-and-cut algorithms [11]. As a consequence, Dantzig–Wolfe decomposition is most effective in settings where block subproblems admit efficient exact or heuristic solvers and where strong stabilization strategies can be employed to control column growth.

Benders Decomposition.

Benders decomposition addresses the block–angular structure by partitioning variables into master variables and subproblem variables [19]. The subproblem variables are eliminated by solving a sequence of subproblems parameterized by the master solution, and their impact is captured through Benders cuts added to the master problem. Unlike the Dantzig–Wolfe decomposition, the classical Benders decomposition converges directly to an optimal solution of the original MILP rather than to a relaxation. Benders decomposition is particularly well suited to problems where the complicating variables appear primarily in constraints rather than in the objective, such as capacity expansion, network design, and two-stage stochastic programming [20, 21]. Numerous extensions, including generalized Benders decomposition and logic-based Benders decomposition, have broadened their applicability to mixed-integer and non-linear settings [22, 23]. Despite its strong theoretical properties, the Benders decomposition can suffer from slow convergence in practice. The number of Benders cuts required to approximate the subproblem value function may grow rapidly, and weak cuts can lead to oscillatory behavior of the master problem. Moreover, the decomposition relies on a clear separation between the master and subproblem variables, which is not always naturally present in highly symmetric or densely coupled formulations.

Lagrangian Relaxation.

Lagrangian relaxation offers a more flexible approach to decomposition by dualizing a selected set of coupling constraints and incorporating them into the objective function with associated multipliers [24]. Relaxing global constraints restores separability across blocks, allowing the resulting Lagrangian subproblem to decompose into independent components. The primary advantage of Lagrangian relaxation lies in its computational simplicity and its ability to generate strong dual bounds for certain classes of problems, particularly in scheduling, routing, and network design [25]. However, because the relaxed problem provides only a bound, additional mechanisms such as subgradient optimization, cutting-plane methods, or heuristic primal recovery are required to obtain feasible integer solutions. Furthermore, the presence of a non-zero duality gap is common in mixed integer settings, limiting the effectiveness of the method as a standalone exact approach.

From Algebraic to Entity-Level Decomposition.

Although algebraic decomposition techniques provide powerful tools for exploiting block–angular structure, their practical deployment often requires substantial problem-specific modeling effort, algorithmic customization, and parameter tuning. In particular, the need for specialized branching strategies, cut management, or multiplier updates can limit their accessibility and robustness in large heterogeneous instances (Rahmaniani et al., 2017 [21]; Barnhart et al., 1998 [11]). These limitations motivate alternative decomposition paradigms that operate at a higher level of abstraction. Rather than decomposing the problem through the algebraic reformulation of constraints, clustering-based approaches decompose the set of entities involved in the optimization, producing a collection of smaller, self-contained subproblems whose sizes can be explicitly controlled, shifting the focus from the exact representation of global coupling to its approximation and subsequent correction. Rather than guaranteeing convergence to optimality through dual coordination, clustering-based methods emphasize scalability, robustness, and the rapid production of high-quality feasible solutions, with residual coupling handled through solution merging and feasibility restoration. The next section formalizes this alternative paradigm and discusses how hierarchical clustering can be used as a principled mechanism for controlling problem dimensions in large-scale MILP formulations.

2.3 Clustering Decomposition and Matheuristic Approaches

Exact algebraic decomposition techniques, most notably Benders decomposition and Dantzig-Wolfe/column generation, offer a principled way to exploit block structure; however, their practical success often hinges on substantial, problem-specific engineering. In many applications, specialized pricing or separation routines, stabilization mechanisms, and branching rules must be designed and then adjusted to obtain reliable convergence and performance [10, 11, 21]. When such overhead becomes prohibitive (or when convergence remains slow despite careful tuning), a common alternative is to decompose the instance *at the entity level*, using clustering to control model size rather than reformulating constraints. This perspective naturally fits within the broader family of *matheuristics*, *i.e.*, hybrid approaches that combine mathematical programming with heuristic or metaheuristic ideas to gain scalability while preserving a structured optimization core [26, 27]. In an entity-level decomposition, the set of decision entities (*e.g.*, customers, routes, OD pairs, facilities, network nodes) is partitioned into clusters; each cluster induces a reduced subproblem that can be solved independently or semi-independently; finally, partial solutions are merged and adjusted to restore feasibility. Unlike exact decomposition, clustering-based methods do not aim to preserve equivalence with the original MILP. Instead, they explicitly trade optimality guaranties for the size of bounded subproblems and practical tractability, making them particularly appealing when the global formulation is

too large or too tightly coupled to be solved reliably within time and memory limits.

The Importance of Size Control.

A large literature on clustering provides many algorithmic options, from partitional methods (*e.g.* k-mean) to graph- and density-based approaches, and hierarchical agglomeration [28, 29]. For decomposition purposes, however, *controlling cluster size is a first-order requirement*: if a clustering method can create arbitrarily large groups, then the computational bottleneck simply reappears inside a few oversized subproblems. This motivates either clustering schemes with explicit size constraints or hierarchical constructions that produce a multiscale tree that can be *cut* to satisfy prescribed size bounds. In practice, the latter is attractive because it yields an entire family of partitions (from coarse to fine) that can be matched to available computational budgets.

Hierarchical Clustering via Neighbor Joining.

Among hierarchical methods, *neighbour joining* (NJ) is a distance-based agglomerative algorithm originally introduced for phylogenetic reconstruction [30]. Given a dissimilarity matrix, NJ builds a binary tree by repeatedly merging the pair that optimizes a global criterion; the resulting hierarchy can then be cut to obtain clusters whose maximum cardinality is bounded by design. Beyond its conceptual simplicity, NJ can be implemented efficiently and has well-studied algorithmic variants, including faster constructions with improved asymptotic complexity [31]. These properties make NJ a convenient choice when the decomposition pipeline requires repeated clustering over large entity sets and when a hierarchical representation is needed to enforce size limits in a principled and reproducible way.

Clustering in Capacitated Optimization Problems.

Clustering is a classical idea in capacitated network and location–allocation settings, especially when the "natural" entities are customers or demands. In vehicle routing, the so-called *cluster-first, route-second* heuristics explicitly group customers into feasible route clusters before optimizing each route in detail; early and influential examples include the sweep algorithm [32] and generalized-assignment-based clustering schemes [33]. The same principle appears in capacitated facility location when demand points are *aggregated* to reduce problem dimension: one solves a smaller location model over aggregated demands and then refines or repairs the solution at the original resolution. Recent work makes this connection explicit by using clustering (*e.g.*, k-means in transformed spaces) as the core mechanism to build high-quality aggregated CFLP instances [34], while previous large-scale studies have already emphasized that heuristic aggregation and relaxation can be decisive for tractability in large facility location instances [35].

Solution Merging and Feasibility Restoration.

A key difficulty of clustering-based decomposition is that independently optimized subproblem solutions do not automatically satisfy global coupling constraints. Consequently, most practical schemes include a dedicated *merging* and *repair* stage, where violations are detected and corrected using targeted re-optimization. A useful viewpoint is to interpret repair as a form of *large neighborhood search*: one "disassembles" parts of the current solution that cause infeasibility and then "repairs" them by solving restricted subproblems, often with MILP as the improvement engine [36, 37]. In the MIP literature, closely related ideas appear in neighborhood-based matheuristics such as Local Branching and RINS, which repeatedly re-solve restricted neighborhoods around incumbents to obtain improvements at moderate cost [38, 39]. In clustering pipelines, analogous mechanisms can be instantiated as (i) local re-optimization over overlapping clusters, (ii) sequential fixing/unfixing of variables associated with violated resources, or (iii) restricted re-solving of the original model with partial fixing to restore feasibility while keeping the optimization scope small.

Positioning of the Proposed Strategy.

The strategy developed in this thesis follows this matheuristic paradigm: clustering is used as a *structural control knob* to limit model dimensions by grouping entities according to their interaction patterns, and a subsequent repair stage addresses the residual coupling induced by shared resources. The distinguishing feature is that mathematical programming remains central at each stage: cluster subproblems are solved to (restricted) optimality, and feasibility restoration is formulated as a targeted optimization task rather than an ad-hoc adjustment. This creates a systematic bridge between structural approximation (through clustering) and robustness (through optimization-based repair), and it provides an interpretable set of parameters to trade solution quality for runtime in large, resource-sharing MILPs.

Chapter 3

Methodology

The strategy proposed in this thesis is meant for large–scale optimization problems where the main computational difficulty is not the lack of local structure, but the presence of *global coupling* induced by *shared limited resources*. In these settings, many decision makers can be treated as largely independent in isolation, but their decisions become strongly interdependent once they compete for the same capacities. This pattern of "many agents, few bottlenecks" is common in networked and service systems and is precisely the regime where controlling model growth through clustering can be beneficial.

3.1 Problem Scope and Applicability Conditions

At a high level, the problems targeted here can be described through three conceptual ingredients:

Entities with Discrete Choices. The instance comprises a set of *entities* (the objects we cluster), such as clients, commodities, origin–destination pairs, or jobs, and each entity must make a decision among a finite set of admissible alternatives. In practice, this often appears as a *restricted-option* representation, where a manageable candidate set is constructed in advance (*e.g.*, a set of feasible routes, a shortlist of eligible facilities, or other enumerated modes). The strategy is designed for problems where such a candidate set can be defined without solving the full global model and where the candidate set is expressive enough to represent competitive behavior around scarce resources.

Shared Capacities that Create Global Coupling. Entities interact because their choices consume capacity from a common pool of resources. These resources may be physical or abstract (space–time network capacity, facility capacity, machine time, shared budget), but the modeling pattern is the same: if many entities are attracted to the same cheap/short alternatives, they overload

the same resources, and feasibility (or performance) becomes a global property. This is the core reason global MILP formulations become difficult at scale: even when the per-entity structure is simple, the global coupling constraints tie together decisions across the entire instance.

A Meaningful Approximation of Interactions. The strategy assumes that the *propensity* of two entities to conflict can be estimated *before* any optimization is performed, using only instance data and the candidate alternatives. Concretely, each alternative induces a resource-usage "footprint" (which resources it touches, and with what intensity), and two entities are strongly interacting if their footprints overlap on scarce resources. The similarity used in the clustering stage is not meant to predict the final optimal assignment; rather, it is meant to identify which entities are *likely to compete for the same bottlenecks* and, therefore, should be solved within the same subproblem.

Within this scope, clustering serves a precise algorithmic purpose: it produces a partition of the entity set into blocks whose size is explicitly bounded, so that the subproblems solved in the cluster phase remain within a controllable computational envelope. This strategy is especially relevant when the global model grows roughly in proportion to the number of entities multiplied by the richness of their candidate alternatives, making the global approach quickly limited in memory or time. In this perspective, merging the resulting cluster solutions can be viewed as a superposition of local plans, where any global inconsistency is largely explained by *overlapping resource usage across clusters*, rather than by incompatible local semantics. Consequently, the resulting infeasibility is not arbitrary but has a clear and localized explanation in terms of *resource overload*. This specific structure implies that violations can be addressed by modifying only a *limited portion* of the merged plan. The repair stage is therefore not an ad–hoc patch, but a controlled mechanism to restore feasibility: it identifies the few resources that become critical after the merge and re-optimizes only the conflicting allocation on residual capacities, thus preserving the value of the decomposed computation without rewriting the entire solution from scratch. The conditions above describe a structural family rather than a single application. Typical domains where this structure arises include network allocation problems with node/arc capacities (often time-indexed), service and logistics models where demand points compete for facility capacities and opening decisions, and scheduling-like settings where tasks compete for shared machines, time windows, or other limited global resources, provided that each entity can be represented through a restricted set of admissible alternatives and that coupling is dominated by shared–capacity effects. The outcome is a feasible global solution obtained by combining bounded-size subproblems solutions with a targeted global reconciliation step. The remainder of this Chapter introduces the pipeline at an algorithmic level using generic notation. The detailed MILP formulations and case study–specific instantiations are deferred to Chapter 5, where the abstract ingredients are specialized into concrete sets, variables, and

constraints. Finally, the presolve indicators are described.

3.2 Proposed Strategy

This section describes the strategy as an algorithmic pipeline, deliberately keeping the presentation problem agnostic. The goal is to make clear (i) what is computed at each stage and (ii) which design parameters control scalability and robustness. Concrete instantiations (similarity footprints, MILP formulations, and repair scoring rules) are specified later in Chapters 4 and 5.

3.2.1 Abstract Components

Let $\mathcal{E}$ be the set of entities denoted by e, and $\mathcal{R}$ be the set of shared resources denoted by r. Each entity $e \in \mathcal{E}$ has a finite set of admissible options $\mathcal{O}_e$ (*e.g.*, paths, facilities, modes) and each resource $r \in \mathcal{R}$ has a capacity $U_r \in \mathbb{R}_+$. A *solution* Sol is understood abstractly as a collection of decision-making at the entity-level, from which resource usage can be derived.

Entity Weight and Cluster Size. The size of the cluster is measured through a nonnegative entity weight $b_e \in \mathbb{Z}_+$[1]: the amount of *atomic units* that must be assigned to entity e (*e.g.* the number of agents, demand units, or simply $b_e \equiv 1$). A cluster $C \subseteq \mathcal{E}$ is feasible with respect to the size limit M if $\sum_{e \in C} b_e \leq M$. This makes the bound meaningful across different case studies.

Residuals and Critical Resources. Given a solution Sol, let $\mathsf{Load}_e(r; Sol)$ denote the total consumption induced on the resource $r \in \mathcal{R}$ by the entity e. Then the total load on resource r is:

$$\mathsf{Load}(r; Sol) \;=\; \sum_{e \in \mathcal{E}} \mathsf{Load}_e(r; Sol) \tag{3.1}$$

We define the *residual* as follows.

$$\mathsf{Res}(r; Sol) \;=\; U_r - \mathsf{Load}(r; Sol) \tag{3.2}$$

A resource is *critical* under tolerance $\tau \geq 0$ if $\mathsf{Res}(r; Sol) < \tau$. When $\tau = 0$, this corresponds to *violated resources* ($\mathsf{Res} < 0$), resources that make the problem infeasible; larger τ also treat near-saturated resources as critical.

Relevant Resources. Let us denote

$$\mathcal{V}^- \;=\; \{\, r \in \mathcal{R} : \mathsf{Res}(r; Sol) < 0 \,\} \tag{3.3}$$

as the set of violated resources. We define the set of entities involved in at least one violation as

$$\mathcal{E}^- \;=\; \{\, e \in \mathcal{E} : \exists r \in \mathcal{V}^- \text{ s.t. } \mathsf{Load}_e(r; Sol) > 0 \,\} \tag{3.4}$$

[1] In some variants, b_e may be continuous, but the pipeline retains the same form.

For each entity e, consider its option set $\mathcal{O}_e$ and let $\mathcal{A}_e(o) \subseteq \mathcal{R}$ denote the *resource footprint* of option $o \in \mathcal{O}_e$, *i.e.*, the set of resources whose load may be affected if e is assigned to option o. The *relevant resources* are then defined as

$$\mathcal{R}_{\text{rel}} \;=\; \bigcup_{e \in \mathcal{E}^-} \; \bigcup_{o \in \mathcal{O}_e} \mathcal{A}_e(o), \tag{3.5}$$

and finally we apply tolerance only locally via

$$\mathcal{R}_{\text{crit}} \;=\; \{\, r \in \mathcal{R} : \mathsf{Res}(r; Sol) < \tau \,\} \;\cap\; \mathcal{R}_{\text{rel}} \tag{3.6}$$

This prevents $\tau > 0$ from promoting globally tight but unrelated resources to critical ones, focusing the repair effort on the neighborhoods of entities that can actually resolve the infeasibility. In what follows, $\mathcal{R}_{\text{crit}}$ is always understood in the filtered sense.

Algorithmic Operators. The main pipeline is expressed through the following black–box operators:

$\mathsf{BuildSimilarity}(\mathcal{I}, \mathcal{E}, \mathcal{R}, \{\mathcal{O}_e\}) \to S$**.** Constructs a symmetric similarity matrix $S \in \mathbb{R}_+^{|\mathcal{E}| \times |\mathcal{E}|}$ from instance data and option–induced resource usage. Larger values indicate stronger anticipated interaction (competition on scarce resources). By convention, $S_{ij} = s(e_i, e_j)$ and $S_{ii} = 0$.

$\mathsf{SimilarityToDistance}(S) \to D$**.** Applies a monotone transformation that converts similarities to a dissimilarity matrix D used for tree construction.

$\mathsf{NeighborJoining}(D) \to Tree$**.** Builds a hierarchical tree whose leaves correspond to entities. Intuitively, entities with smaller pairwise distances are merged earlier, producing a dendrogram–like structure.

$\mathsf{ExtractPartition}(Tree, M, \{b_e\}_{e \in \mathcal{E}}) \to \mathcal{P} = \{C_1, \ldots, C_m\}$**.** Extracts a partition from the tree such that each cluster satisfies the size limit $\sum_{e \in C_k} b_e \le M$. Operationally, this is a tree–cutting procedure guided by the size bound.

$\mathsf{SolveCluster}(C_k, \mathcal{I}) \to Sol_k$**.** Solves the cluster–restricted optimization subproblem for entities in C_k (with the original resources/capacities), producing a locally coherent plan Sol_k for those entities.

$\mathsf{MergeSolutions}(\{Sol_k\}, \mathcal{I}) \to Sol^{merge}$**.** Combines cluster solutions into a global draft solution. Conceptually, this is the superposition of entity decisions.

$\mathsf{IsFeasible}(Sol, \mathcal{I}) \to \{\texttt{true}, \texttt{false}\}$**.** Checks global feasibility of Sol with respect to the full constraint set.

$\mathsf{RepairAndReducedSolve}(Sol^{merge}, \mathcal{I}, \mathcal{R}, \tau) \to Sol^{final}$**.** Restores feasibility by selectively releasing a small portion of the merged plan around critical resources and re–optimizing only the released part on residual capacities.

3.2.2 Main Pipeline

The Algorithm 1 presents the pipeline as a compact blueprint. The first block constructs a bounded-size partition from presolve similarity information; the second block solves one model per cluster and merges results; the final block invokes repair only if the merged draft violates global constraints.

Algorithm 1: Main Pipeline

Input: $\mathcal{I}$; $\mathcal{E}$; $\mathcal{R}$; $\{\mathcal{O}_e\}_{e\in\mathcal{E}}$; M; $\{b_e\}_{e\in\mathcal{E}}$; τ
Output: Sol^{final}
$S \leftarrow \mathsf{BuildSimilarity}(\mathcal{I}, \mathcal{E}, \mathcal{R}, \{\mathcal{O}_e\})$;
$D \leftarrow \mathsf{SimilarityToDistance}(S)$;
$Tree \leftarrow \mathsf{NeighborJoining}(D)$;
$\mathcal{P} \leftarrow \mathsf{ExtractPartition}(Tree, M, \{b_e\}_{e\in\mathcal{E}})$;
foreach $C_k \in \mathcal{P}$ **do**
  $Sol_k \leftarrow \mathsf{SolveCluster}(C_k, \mathcal{I})$;
end
$Sol^{merge} \leftarrow \mathsf{MergeSolutions}(\{Sol_k\}_{k=1}^{m}, \mathcal{I})$;
if $\mathsf{IsFeasible}(Sol^{merge}, \mathcal{I})$ **then**
  return Sol^{merge};
end
$Sol^{final} \leftarrow \mathsf{RepairAndReducedSolve}(Sol^{merge}, \mathcal{I}, \mathcal{R}, \tau)$;
return Sol^{final};

3.2.3 Feasibility restoration

The repair stage is designed for the intended scope, where infeasibility after merging cluster solutions is mainly explained by *localized resource overload*. It therefore acts only on a limited set of decisions near the most critical bottlenecks. To express this generically, we assume that the solution can be partially *released* at some application–dependent granularity: a release unit may be an entire entity (*e.g.*, a client) or an atomic portion of an entity (*e.g.*, a single agent, a client demand unit). We denote by ρ the set of released units that will be re–optimized, while the remainder remains fixed. The algorithm 2 implements a two-level loop: (i) at a fixed tolerance τ', repeatedly releasing units until no resource is critical; (ii) solving a reduced optimization problem that reallocates only the released units on the residual capacities. If infeasibility persists, increase τ' and restart, forcing a stronger release.
The last four abstract operators are the following.

$\mathsf{Candidates}(Sol^{fixed}, r^\star, \mathcal{I}) \to \Phi_{r^\star}$**.** Selects which parts of the current plan are allowed to be released (*e.g.*, entities touching $r^\star$, or atomic units currently assigned through $r^\star$).

Algorithm 2: Feasibility Restoration

Input: Sol^{merge}; $\mathcal{I}$; $\mathcal{R}$; τ
Output: Sol^{final}
$\tau' \leftarrow \tau$; $Sol^{fixed} \leftarrow Sol^{merge}$; $\rho \leftarrow \emptyset$
while true **do**
  $\{\mathrm{Res}(r; Sol^{fixed})\}_{r\in\mathcal{R}} \leftarrow \mathsf{ComputeResiduals}(Sol^{fixed}, \mathcal{I}, \mathcal{R})$
  $\mathcal{R}^{crit} \leftarrow \{r \in \mathcal{R} : \mathrm{Res}(r; Sol^{fixed}) < \tau'\} \cap \mathcal{R}_{\mathrm{rel}}$ aggiungere la condizione di stop
  while $\mathcal{R}^{crit} \neq \emptyset$ **do**
    $r^\star \leftarrow \arg\min_{r\in\mathcal{R}^{crit}} \mathrm{Res}(r; Sol^{fixed})$
    $\Phi_{r^\star} \leftarrow \mathsf{Candidates}(Sol^{fixed}, r^\star, \mathcal{I})$
    foreach $\phi \in \Phi_{r^\star}$ **do**
      $\sigma_\phi \leftarrow \mathsf{Score}(\phi, \mathcal{I})$
    end
    $\phi^\star \leftarrow \arg\max_{\phi\in\Phi_{r^\star}} \sigma_\phi$
    $(Sol^{fixed}, \Delta^{rel}) \leftarrow \mathsf{Release}(Sol^{fixed}, \phi^\star, \mathcal{I})$
    $\rho \leftarrow \rho \cup \Delta^{rel}$
    $\{\mathrm{Res}(r; Sol^{fixed})\}_{r\in\mathcal{R}} \leftarrow \mathsf{ComputeResiduals}(Sol^{fixed}, \mathcal{I}, \mathcal{R})$
    $\mathcal{R}^{crit} \leftarrow \{r \in \mathcal{R} : \mathrm{Res}(r; Sol^{fixed}) < \tau'\} \cap \mathcal{R}_{\mathrm{rel}}$
  end
  $Sol^{red} \leftarrow \mathsf{SolveReduced}\left(\rho, Sol^{fixed}, \mathcal{I}\right)$
  if $\mathsf{IsFeasible}\left(Sol^{red}, \mathcal{I}\right)$ **then**
    $Sol^{final} = Sol^{fixed} \cup Sol^{red}$
    return Sol^{final}
  end
  $\tau' \leftarrow \tau' + 1$
end

$\mathsf{Score}(\phi, \mathcal{I}) \rightarrow \sigma_\phi$**.** Ranks candidates $\phi \in \Phi_{r^\star}$ using a model–dependent proxy for "benefit" (*e.g.*, expected reduction of overload, ability to reroute elsewhere, or minimal disruption).

$\mathsf{Release}(Sol^{fixed}, \phi^\star, \mathcal{I}) \rightarrow (Sol^{fixed}, \Delta^{rel})$**.** Release an amount of units for candidate $\phi^\star$, modifing the fixed solution.

$\mathsf{SolveReduced}(\rho, Sol^{fixed}, \mathcal{I}) \rightarrow Sol^{red}$**.** Is a reduced optimization model that reassigns only ρ given the fixed remainder.

Key Design Parameters

The strategy behavior is governed by three main parameters that control the critical trade–off between execution time and the level of global coordination. Specifically:

- The cluster size bound (M). This parameter directly dictates the complexity of the subproblems. A larger M allows for the internalization of

more conflicts, potentially improving solution quality, but increases the computational cost of solving individual clusters. In contrast, a small M ensures speed, but increases the risk of infeasibility between clusters.

- The similarity and footprint design. This definition determines the topology of the partition. It defines what constitutes an "interaction" and directs the clustering algorithm to group specific entities. A well–designed similarity metric minimizes the residual coupling left for the repair stage.
- The repair tolerance (τ). This parameter controls the aggressiveness of the restoration phase. In this thesis, we always start from $\tau = 0$.

In the following chapters, these degrees of freedom are instantiated and evaluated in selected case studies.

3.3 Presolve Indicators

This thesis introduces a set of *presolve indicators* computed immediately after the similarity matrix and the size–constrained partition are available, and *before* solving any cluster–restricted optimization model. Their role is descriptive: they summarize, in a compact and interpretable way, how the chosen clustering configuration organizes the interaction structure implied by the similarity, and how much *residual coupling* is still expected to remain across clusters due to shared resources. Importantly, these indicators do not attempt to forecast solver outcomes in a statistical sense, nor to certify that a given configuration will succeed. Instead, they provide a principled lens to compare alternative settings (*e.g.*, different maximum cluster sizes, option sets, or similarity definitions) and to identify configurations that appear internally coherent versus configurations that expose strong cross–cluster interaction already at the presolve level.

3.3.1 Similarity Indicators

The first family of indicators quantifies how much of the total similarity mass is retained within clusters. Define the total pairwise similarity mass as

$$\mathrm{SIM}_{\mathrm{tot}} \;=\; \sum_{i<j} S_{ij} \tag{3.7}$$

For each cluster $C \in \mathcal{P}$ define its intra–cluster similarity mass as

$$\mathrm{SIM}_{\mathrm{intra}}(C) \;=\; \sum_{\substack{i<j \\ e_i,e_j \in C}} S_{ij} \tag{3.8}$$

and its cross–cluster similarity mass as

$$\mathrm{SIM}_{\mathrm{cross}}(C) \;=\; \sum_{\substack{e_i \in C \\ e_j \notin C}} S_{ij} \tag{3.9}$$

Global Similarity Index. We define the *similarity index* as the fraction of the total similarity mass that lies within clusters:

$$R_{\text{sim}} \;=\; \frac{\sum_{C\in\mathcal{P}} \text{SIM}_{\text{intra}}(C)}{\text{SIM}_{\text{tot}}} \;\in\; [0,1] \tag{3.10}$$

Values close to 1 indicate that the partition captures the most strong interactions internally, whereas lower values indicate that a substantial fraction of the similarity mass is divided across clusters.

Cluster Similarity Index. To localize this information, we also define the ratio for each group C

$$R_{\text{sim}}(C) \;=\; \frac{\text{SIM}_{\text{intra}}(C)}{\text{SIM}_{\text{intra}}(C) + \text{SIM}_{\text{cross}}(C)} \;\in\; [0,1] \tag{3.11}$$

This quantity is high when the interactions *involving* C are mostly internal to C, and low when C remains strongly tied to entities outside the cluster. In practice, $R_{\text{sim}}(C)$ helps identify clusters that are structurally self–contained versus clusters that act as "bridges" in the interaction graph.

Minimum Cluster Similarity Index. Finally, to capture the worst–case cohesiveness, we consider the minimum of the cluster similarity indices across the partition:

$$\widehat{R}_{\text{sim}} \;=\; \min_{C\in\mathcal{P}} R_{\text{sim}}(C) \tag{3.12}$$

This indicator serves as a bottleneck detector: while the global index R_{sim} provides an aggregate view, $\widehat{R}_{\text{sim}}$ reveals if there exists at least one cluster that fails to be sufficiently self–contained. A low value suggests that, despite a potentially high overall quality of the partition, specific parts of the decomposition remain strongly coupled with the rest of the instance.

Max–Mean Inter–Cluster Similarity Index. Although the previous indices focus on retention of similarity within clusters, it is equally important to detect if specific pairs of clusters remain strongly coupled. To this end, for any pair of distinct clusters $C_a, C_b \in \mathcal{P}$, we first define the *mean inter–cluster similarity* as the average similarity between an entity in C_a and an entity in C_b:

$$\text{SIM}_{\text{mean}}(C_a, C_b) \;=\; \frac{1}{|C_a| \cdot |C_b|} \sum_{e_i\in C_a} \sum_{e_j\in C_b} S_{ij} \tag{3.13}$$

Using this measure, we define the indicator $\overline{R}_{\text{sim}}$ as the maximum mean similarity observed across all pairs:

$$\overline{R}_{\text{sim}} \;=\; \max_{\substack{C_a,C_b\in\mathcal{P}\\ C_a\neq C_b}} \text{SIM}_{\text{mean}}(C_a, C_b) \tag{3.14}$$

Unlike previous indicators where higher values were generally desirable, here a high value of $\overline{R}_{\text{sim}}$ is unfavorable. It signals the presence of at least two specific clusters that share strong average connectivity, suggesting that they might be candidates for merging or that the partition locally forces a separation between strongly related subgroups.

Note on Standard Clustering Metrics

A ubiquitous metric in cluster analysis is the *Silhouette Coefficient* [40], often used to assess the quality of a partition by measuring how similar an object is to its own cluster (cohesion) compared to other clusters (separation). Formally, assuming a distance metric $d(e_i, e_j)$ inversely related to similarity S_{ij}, the silhouette value $\overline{s}(\mathcal{P})$ is defined as:

$$\overline{s}(\mathcal{P}) = \frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} \frac{d_{out}(e, \mathcal{P}) \; - \; d_{in}(e, \mathcal{P})}{\max\left(d_{out}(e, \mathcal{P}), \; d_{in}(e, \mathcal{P})\right)} \tag{3.15}$$

where $d_{in}(e, \mathcal{P})$ is the mean distance between e and all other points in the same cluster, and $d_{out}(e, \mathcal{P})$ is the minimum mean distance from e to points in any other cluster:

$$d_{out}(e, \mathcal{P}) = \min_{C \in \mathcal{P} : C \not\ni e} \min_{e' \in C} d(e, e') \tag{3.16}$$

$$d_{in}(e, \mathcal{P}) = \frac{1}{|C \in \mathcal{P} : C \ni e| - 1} \sum_{e' \in C \ni e \; \wedge \; e \neq e'} d(e, e') \tag{3.17}$$

Note that $\overline{s}(\mathcal{P}) \in [-1, 1]$: the larger (closer to 1), the better (*i.e.*, the more separated the clusters). Despite its popularity, we explicitly exclude the Silhouette Coefficient from our set of indicators. The rationale lies in the fundamental difference between clustering for *data exploration* and clustering for *problem decomposition*. Standard metrics like Silhouette penalize clusters that lack internal compactness. However, our proposed neighbor joining approach, driven by a strict maximum capacity constraint, acts effectively as a bin–packing heuristic in its final stages. It may aggregate structurally disjoint components into a single cluster to maximize the utilization of the available capacity. Specifically, multiple small, unrelated subgraphs resulting from the hierarchical cut of the phylogenetic tree may be grouped together simply because their combined size respects the maximum limit. In such cases, the intra–cluster distance $d_{in}(e, \mathcal{P})$ would be large, resulting in a poor Silhouette score. Yet, from a decomposition perspective, this aggregation is valid: placing independent sub–problems into the same container does not introduce new cross–cluster dependencies. Therefore, a low Silhouette score would be misleading, flagging these heterogeneous but non–interacting clusters as defects.

3.3.2 Congestion Indicators

Retention of similarity alone does not capture *capacity stress*. The second family of indicators measures how the partition interacts with shared resource capacities under a conservative notion of *potential load*.

Potential Occupancy. Let $\boldsymbol{a_e}^{(r)}$ denote the potential contribution of entity e to the load of resource r, formally defined in Section 4.1.2. For each subset $X \subseteq \mathcal{E}$ and each resource $r \in \mathcal{R}$, we define the occupancy as

$$\mathrm{occ}_X(r) \;=\; \sum_{e \in X} b_e \, \boldsymbol{a_e}^{(r)}. \tag{3.18}$$

Conceptually, $\mathrm{occ}_X(r)$ aggregates the average resource usage induced by the options of entities in X and scales it by an entity "mass" (*e.g.*, the number of agents for OD pairs or the demand for clients). In the implementation used for experiments, each entity spreads its mass evenly across its candidate options, and the resulting contributions are accumulated per resource.

Absolute Overload and Maximum Ratio. Given $\mathrm{occ}_X(\cdot)$, define the *absolute overload* of a subset X as

$$E_{\mathrm{abs}}(X) \;=\; \sum_{r \in \mathcal{R}} \big(\mathrm{occ}_X(r) - U_r\big)_+, \qquad \text{where } (y)_+ := \max\{0, y\} \tag{3.19}$$

and the *maximum congestion ratio* as

$$R_{\max}(X) \;=\; \max_{r \in \mathcal{R}} \frac{\mathrm{occ}_X(r)}{U_r} \tag{3.20}$$

with the convention that resources with zero capacity are treated as infinitely tight. Absolute overload captures *how much* capacity is exceeded in total (in the same units as occupancy), while $R_{\max}$ isolates the single worst bottleneck. Both are computed at the *cluster level* ($X = C_k$) and at the *global* level ($X = \mathcal{E}$). For the sake of concise notation, we denote the global metrics simply as

$$E_{\mathrm{abs}} \equiv E_{\mathrm{abs}}(\mathcal{E}), \quad R_{\max} \equiv R_{\max}(\mathcal{E}) \tag{3.21}$$

and the maximum values recorded across any single cluster in the partition as

$$\overline{E}_{\mathrm{abs}} \;=\; \max_{C \in \mathcal{P}} E_{\mathrm{abs}}(C), \quad \overline{R}_{\max} \;=\; \max_{C \in \mathcal{P}} R_{\max}(C) \tag{3.22}$$

Normalized Congestion Index. Because instances may differ in total scale, we also report a normalized overload per unit of total mass:

$$E_{\mathrm{norm}}(X) \;=\; \frac{E_{\mathrm{abs}}(X)}{\mathsf{mass}(X)} \tag{3.23}$$

where

$$\mathsf{mass}(X) = \sum_{e \in X} b_e \tag{3.24}$$

This yields a dimensionless quantity that is comparable across clusters and instances. Analogously to the previous metrics, we denote the global normalized index as

$$E_{\text{norm}} \;\equiv\; E_{\text{norm}}(\mathcal{E}) \tag{3.25}$$

and the peak normalized congestion among all clusters as

$$\overline{E}_{\text{norm}} \;=\; \max_{C \in \mathcal{P}} E_{\text{norm}}(C) \tag{3.26}$$

Cross–Cluster Congestion Indicators

The previous indicators can be combined to isolate the portion of overload that is attributable to *cross–cluster* interaction. Let $E_{\text{abs}}(\mathcal{E})$ be the global overload calculated for the entire set of entities, and let $\sum_{k=1}^{m} E_{\text{abs}}(C_k)$ be the sum of overloads that would be observed if each cluster were considered in isolation. We define the *cross–cluster overload* as

$$E_{\text{cross}} \;=\; \left(E_{\text{abs}}(\mathcal{E}) - \sum_{k=1}^{m} E_{\text{abs}}(C_k) \right)_{+} \tag{3.27}$$

Intuitively, E_{cross} captures the additional overload observed when moving from isolated cluster views to the full combined view, under the same expected occupancy model, *i.e.*, the portion of capacity stress that cannot be explained by within–cluster congestion alone.
Normalized views of the same quantity are also reported:

$$R_{\text{cross}} \;=\; \frac{E_{\text{cross}}}{\mathsf{mass}(\mathcal{E})} \tag{3.28}$$

Implementation note. All indicators above are computed as part of the clustering step: once the similarity matrix is built and the NJ partition is extracted, expected occupancies and the resulting congestion metrics are evaluated, followed by the similarity concentration metrics. This ensures that the indicators are available before any optimization subproblem is solved, making them suitable for comparing configurations at negligible additional computational cost. Furthermore, these indicators must be considered probabilistic quantities induced by the option sets and hence conservative and model-dependent.

Chapter 4

Similarity Modeling and Neighbor Joining Clustering

The strategy introduced in the previous chapter describes the decomposition process through a sequence of abstract operators, deliberately deferring their implementation details to maintain a general perspective. This chapter provides a concrete realization of the first block of that pipeline: *similarity computation* and *cluster decomposition*. We specifically focus on instantiating the abstract ingredients responsible for analyzing the instance structure—namely the functions BuildSimilarity and NeighborJoining. In the following sections, we detail how the generic concept of "interaction" is mapped into a quantitative similarity metric and how the Neighbor Joining algorithm is adapted to strictly enforce the cluster size bounds required for computational control.

4.1 Similarity Function Design

This section details the construction of the similarity metric S, formally realizing the abstract BuildSimilarity operator. The purpose of the similarity function is not to approximate the objective value or to directly predict the final assignment decisions. Rather, it aims to approximate *the expected coupling* between entities induced by shared resources and global constraints. In resource–sharing MILPs, coupling is the main obstacle to effective decomposition: separating strongly interacting entities tends to create infeasibilities after merging cluster solutions and increases the burden of feasibility restoration. The similarity function is therefore designed as a structural proxy for *how likely two entities are to compete for the same limited capacities*.

4.1.1 Design Principles

For any pair of entities $e, e' \in \mathcal{E}$, we seek a similarity score $s(e, e')$ that serves as a robust structural proxy for potential conflict. To ensure that the resulting decomposition is meaningful and computationally viable, the metric is designed to satisfy the following set of theoretical and practical properties.

Symmetry and Nonnegativity. Since similarity represents a strength of mutual interaction, we inherently require $s(e, e') = s(e', e)$ and $s(e, e') \geq 0$.

Resource-Aware Interpretation. The score should be large precisely when e and e' are expected to load the same scarce resources. Unlike simple geometric proximity, this metric must capture *functional coupling*: a high similarity implies a high probability that the two entities will compete for the same limited capacities in the final allocation. This interpretability is crucial for motivating the clustering outcomes.

Robustness to Noise. Candidate option sets are often generated heuristically (*e.g.*, restricted path generation). The similarity should be stable under small perturbations of $\mathcal{O}_e$; overlap-based footprints with mild option weights π_e help achieve this.

Presolve Efficiency. The metric must be computable strictly from instance data and the restricted option sets, without requiring the solution of any optimization model. Furthermore, since computing a dense interaction matrix scales as $O(|\mathcal{E}|^2)$, the definition should allow efficient implementation (*e.g.*, exploiting sparsity or approximation) to avoid dominating the runtime in large-scale instances [41, 42]. The key modeling choice to satisfy these principles is the definition of a *resource footprint* for each entity, which acts as the fundamental building block from which pairwise similarities are derived.

4.1.2 A General Footprint Template

Assume that each entity $e \in \mathcal{E}$ has a finite set of options $\mathcal{O}_e$. Each option $o \in \mathcal{O}_e$ induces the usage of a subset of resources in $\mathcal{R}$. We define a nonnegative footprint vector $\boldsymbol{a_e} \in \mathbb{R}_+^{|\mathcal{R}|}$ with components

$$\boldsymbol{a_e}^{(r)} \quad = \quad \sum_{o \in \mathcal{O}_e} \pi_e(o)\, u_{e,o}(r) \qquad r \in \mathcal{R}, \tag{4.1}$$

where:

- $\pi_e(o) \in \mathbb{R}_+$ is a presolve weighting of options (*e.g.* uniform over candidate paths; or a bias towards cheaper/shorter options if appropriate).
- $u_{e,o}(r) \in \mathbb{R}_+$ quantifies how option o uses resource r (often binary: uses/does not use; or a multiplicity such as time spent);

This template deliberately separates (i) *what the entity could do* (its option set) from (ii) *how it is expected to load resources* (the footprint). The footprint does not need to be probabilistically correct; it must be a stable proxy for where coupling can arise.

4.1.3 Similarity Families

Given $\boldsymbol{a_e}$ for each $e \in \mathcal{E}$, we use similarities that increase with overlap of the footprint. Two families are particularly appropriate in resource-sharing settings:

Overlap (Inner–Product) Similarity.

$$s_{\mathrm{ov}}(e, e') \;=\; \zeta(e, e') \sum_{r \in \mathcal{R}} w(r)\, \boldsymbol{a_e}^{(r)}\, \boldsymbol{a_{e'}}^{(r)} \tag{4.2}$$

where $w(r) \geq 0$ are optional resource weights and $\zeta(e, e') \geq 0$ are optional entity weights that involve atomic units of entities. This form emphasizes *co–loading*: if both entities are large on the same resources, similarity grows rapidly. It is well suited when congestion risk is driven by simultaneous usage (such as in time-indexed capacities).

Normalized Overlap (Cosine) Similarity.

$$s_{\cos}(e, e') \;=\; \zeta(e, e') \frac{\sum_{r \in \mathcal{R}} w(r)\, \boldsymbol{a_e}^{(r)}\, \boldsymbol{a_{e'}}^{(r)}}{\sqrt{\sum_{r \in \mathcal{R}} w(r)\, \boldsymbol{a_e}^{(r)^2}}\, \sqrt{\sum_{r \in \mathcal{R}} w(r)\, \boldsymbol{a_{e'}}^{(r)^2}}} \tag{4.3}$$

The cosine similarity is invariant with the positive scaling of $\boldsymbol{a_e}$, so it captures alignment rather than magnitude. It is useful when demands are highly heterogeneous and one wants clusters to reflect interaction structure rather than size alone.

In this thesis, we choose the following.

- uniform option weights $\pi_e(o) = \dfrac{1}{|\mathcal{O}_e|}$ to avoid injecting a priori preferences among candidate options.
- overlap–based similarity because it directly encodes the expected stress on shared capacities.
- $w(r)$ conservative: by default $w(r) = 1$.
- $\zeta(e, e') = \min(b_e, b_{e'})$ because the potential resource contention between two entities is naturally bounded by the size of the smaller one. This captures the *effective* volume of conflict without disproportionately biasing the clustering towards large entities based on size alone.

So, the similarity between the entities e and e' is:

$$s(e, e') \;=\; \min\{b_e, b_{e'}\} \sum_{r \in \mathcal{R}} \left(\frac{1}{|\mathcal{O}_e|} \sum_{o \in \mathcal{O}_e} u_{e,o}(r) \right) \left(\frac{1}{|\mathcal{O}_{e'}|} \sum_{o' \in \mathcal{O}_{e'}} u_{e',o'}(r) \right) \quad (4.4)$$

4.1.4 From Similarities to Distances

NJ requires a dissimilarity matrix. Since our similarity is designed as an interaction score, we convert it into a distance through a monotone transformation

$$d(e, e') \;=\; g(s(e, e'))\,, \quad (4.5)$$

where $g(\cdot)$ is strictly decreasing. In this thesis, we adopt a transformation that preserves pairwise rankings while ensuring nonnegative distances and numerical stability.

$$d(e, e') := \begin{cases} s_{\max} + 1 - s(e, e') & e \neq e' \\ 0 & e = e' \end{cases}, \quad s_{\max} = \max_{i \neq j} s(i, j) \quad (4.6)$$

formally realizing the SimilarityToDistance operator.

The next section details how these distances drive a constrained hierarchical clustering process, thereby completing the instantiation of the decomposition phase introduced in Section 3.2. Specifically, we define the concrete logic for the NeighborJoining and ExtractPartition operators.

4.2 Neighbor Joining for Constrained Hierarchical Clustering

In biology, a *taxon* (plural: *taxa*) is a group of one or more populations of an organism, or organisms, as seen by taxonomists to form a biological unit. For us, a taxon is simply an entity. In bioinformatics, NJ [30] is a bottom-up (agglomerative) clustering method for the creation of phylogenetic trees, a branching diagram or tree showing the evolutionary relationships among various biological species based on similarities and differences in their physical or genetic characteristics. The algorithm requires knowledge of the distance between each pair of taxa to create the phylogenetic tree [43]. In this thesis, NJ is used purely as an algorithmic scaffold: the tree provides a compact, global representation of the interaction structure implied by the distances. To obtain a suitable decomposition for the cluster phase, we convert the NJ tree into an explicit partition under a maximum cluster-size constraint. This requires a deterministic cutting strategy that splits oversized subtrees and handles unbalanced branches while preserving reproducibility via explicit tie-breaking rules. Finally, we discuss computational aspects of NJ and the practical implementation choices that make it tractable at the instance sizes considered.

4.2.1 The Algorithm

Neighbor Joining takes as input the *distance matrix* D derived in Section 4.1, which quantifies the pairwise dissimilarity between entities. The algorithm is initialized with a star-like topology, representing a completely unresolved tree. It then proceeds in an agglomerative fashion, iteratively joining pairs of nodes until the structure is fully resolved into a binary tree and all branch lengths are determined. In any iteration, let V denote the set of currently active nodes (initially $V = \mathcal{E}$) and let $n = |V|$.

The algorithm proceeds as follows:

1. Based on the current distance matrix, calculate the Q–matrix.
2. Find the pair of distinct taxa $(i^\star, j^\star)$ (*i.e.*, with $i^\star \neq j^\star$) for which Q is the smallest.
3. Make a new node u that joins the taxa $i^\star$ and $j^\star$, and connect u to the central node.
4. Calculate the distance from each of the taxa in the pair to this new node.
5. Calculate the distance from each of the taxa outside of this pair to the new node.
6. Update $V \leftarrow (V \setminus \{i^\star, j^\star\}) \cup \{u\}$ and repeat the process until $|V| = 3$, at which point the tree is completed by connecting the last three nodes.

The Q–Matrix. Based on D relating to the n taxa, calculate the $n \times n$ Q–matrix as follows:

$$Q_{i,j} = (n-2)D_{i,j} - \sum_{k \in V} D_{i,k} - \sum_{k \in V} D_{j,k} \tag{4.7}$$

Distance From the Pair Members To the New Node. For each of the taxa in the pair that being joining, use the following formula to calculate the distance to the new node:

$$\delta(i^\star, u) = \frac{1}{2} D_{i^\star, j^\star} + \frac{1}{2(n-2)} \left[\sum_{k \in V} D_{i^\star, k} - \sum_{k \in V} D_{j^\star, k} \right] \tag{4.8}$$

$$\delta(j^\star, u) = D_{i^\star, j^\star} - \delta(i^\star, u) \tag{4.9}$$

Distance of the Other Taxa From the New Node For each taxon $k \in V \setminus \{i^\star, j^\star\}$ not considered in the previous step, we calculate the distance to the new node as follows:

$$D_{u,k} = \frac{1}{2} \Big[D_{i^\star, k} + D_{j^\star, k} - D_{i^\star, j^\star} \Big] \tag{4.10}$$

An example of the NJ algorithm is shown in Figure 4.1.

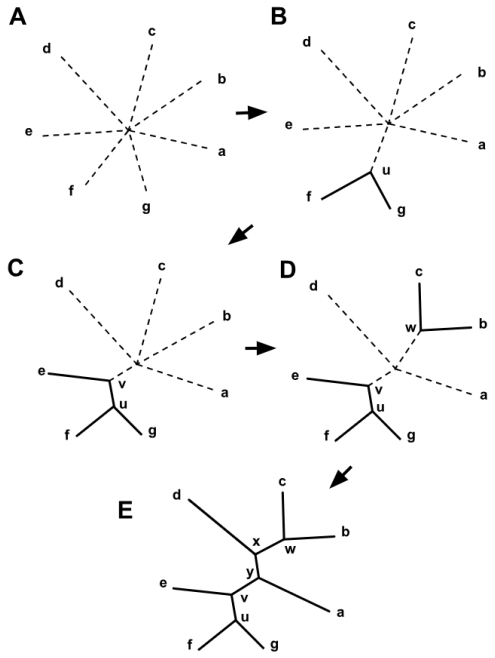


Figure 4.1: Starting with a star tree (A), the Q matrix is calculated and used to choose a pair of nodes for joining, in this case f and g. These are joined to a newly created node, u, as shown in (B). The part of the tree shown as solid lines is now fixed and will not be changed in subsequent joining steps. The distances from node u to the nodes a-e are computed from equations (4.8)–(4.9). This process is then repeated, using a matrix of just the distances between the nodes, a,b,c,d,e, and u, and a Q matrix derived from it. In this case u and e are joined to the newly created v, as shown in (C). Two more iterations lead first to (D), and then to (E), at which point the algorithm is done, as the tree is fully resolved.

4.2.2 Computational Aspects

A straightforward implementation recomputes Q at each iteration and searches its minimum on all pairs, leading to $O(n^3)$ time [44] and $O(n^2)$ memory in the number of initial entities $n = |\mathcal{E}|$. Several implementations accelerate the search for the minimizing pair and can achieve near-quadratic behavior in practice while preserving the canonical NJ criterion (*e.g.* RapidNJ [45]), and related variants (*e.g.* Fast Neighbor Joining [46]) provide worst-case improvements un-

der modified selection strategies. In our setting, the Q-matrix computation is amenable to vectorization, which is relevant because distance construction and clustering are performed as presolve steps and must remain lightweight compared to MILP optimization.

4.2.3 Extracting a Partition

While the Neighbor Joining algorithm outputs a hierarchical binary tree, the decomposition pipeline requires an explicit partition of the entity set into computationally manageable blocks. To enforce this, we utilize the entity weight b_e (introduced in the previous section as the *size contribution* of e). Whether b_e represents atomic units, demand, or simply cardinality ($b_e \equiv 1$), we impose that every cluster C in the final partition must satisfy $\mathsf{mass}(C) \leq M$, using the definition of mass provided in Section 3.3. We obtain a size-feasible partition from the NJ tree via a greedy contraction procedure driven by *cherries*. A cherry is an internal node adjacent to two leaves; it represents the most local merge candidate in the current tree. The procedure iteratively contracts feasible cherries while preventing the creation of oversized aggregates:

1. Initialize all leaves as singleton clusters with size b_e.
2. Maintain the set of cherries in the current tree and associate each cherry with the tentative size obtained by merging its two leaf neighbors.
3. Select the cherry with the minimum tentative size (ties are broken deterministically by the fixed order of internal nodes in the NJ construction).
4. If the tentative size is $\leq M$, *accept* the merge: contract the cherry into a new leaf representing the merged cluster and update the set of cherries locally.
5. Otherwise, *reject* the merge by pruning this local configuration so that the two leaf substructures are not forced to merge through that internal node; update the tree connectivity, and continue.

The acceptance step guaranties that every created cluster respects the size limit by construction. The rejection (pruning) step is the key mechanism for handling unbalanced branches: when a local merge would exceed M, the algorithm modifies the local tree structure so that the involved subtrees can still be clustered through alternative, smaller merges elsewhere in the hierarchy. At termination, a small residual tree remains; a final pass may optionally merge the lightest remaining clusters whenever their combined size stays within M, reducing fragmentation without violating the constraint.

Chapter 5

Mathematical Models

This chapter completes the instantiation of the pipeline introduced in Chapters 3 and 4. While the previous chapter focused on the construction of the similarity matrix and the partition strategy, this chapter addresses the optimization and repair phases. Specifically, we provide concrete realizations of the abstract operators defined in Sections 3.2.1 and 3.2.3, applying them to two distinct problem families. Furthermore, we specify the $u_{e,o}(r)$ function (introduced in 4.1.2) for these contexts, thus fully determining the $occ_X(r)$ function presented in Section 3.3. To ensure a coherent presentation, this chapter is organized hierarchically. We begin by establishing a general mathematical framework which formalizes the shared structure underlying both applications. Subsequently, we instantiate this template for two specific case studies, detailing how the generic concepts map onto the actual constraints and variables of each domain.

5.1 A General Mathematical Framework

To ensure a unified treatment of the diverse applications considered in this thesis, we abstract their common features into a general MILP structure. Although the semantic meaning of an entity or a resource varies between domains, the underlying mathematical relationship (discrete choice allocation under shared capacity constraints) remains identical. This abstraction allows us to define the optimization, merging, and repair mechanics strictly once, avoiding redundancy in the mathematical formulation. We adopt the generic notation introduced in Sections 3.2 $(\mathcal{E}, \mathcal{R}, \mathcal{O}_e, U_r, b_e)$ and 4.1 $\left(\boldsymbol{a_e}^{(r)}, u_{e,o}(r)\right)$. This section completes the instantiation of the pipeline introduced in Chapters 3 and 4. While the previous chapter focused on the construction of the similarity matrix and the partition strategy, this chapter addresses the optimization and repair phases. Specifically, we provide concrete realizations of the abstract operators defined in Sections 3.2.1 and 3.2.3, applying them to two distinct problem families. Furthermore, we introduce the $u_{e,o}(r)$ function defined in 4.1.2, which fully determines the $occ_X(r)$ function described in Section 3.3.To ensure a coherent

presentation, this chapter is organized hierarchically. We begin by establishing a general mathematical framework which formalizes the shared structure underlying both applications. Subsequently, we instantiate this template for two specific case studies, detailing how the generic concepts map onto the actual constraints and variables of each domain.ons, we formalize the input parameters, the global MILP template, the subproblem definitions that constitute the core of the computational pipeline, and the feasibility restoration.

Sets and Parameters

For each entity $e \in \mathcal{E}$, we define $c_{e,o} \in \mathbb{R}_+$ as the cost incurred per unit assigned to option $o \in \mathcal{O}_e$ (*e.g.*, delay in travel with respect to the shortest path, assignment cost).
Optionally, some models include activation/opening decisions:

- $\mathcal{Y} \subseteq \mathcal{R}$: a set of *activatable* resources
- $F_r \in \mathbb{R}_+$: the fixed cost of activating $r \in \mathcal{Y}$.

Decision Variables

- $x_{e,o} \geq 0$: the amount of entity e assigned to option $o \in \mathcal{O}_e$. Depending on the case study, $x_{e,o}$ is an integer (indivisible atomic units) or a continuous (splittable demand).
- $z_r \in \{0, 1\}$ (optional): activation variable of resource $r \in \mathcal{Y}$.

When activation variables are present, the effective capacity of a resource $r \in \mathcal{Y}$ is typically modeled as a function $\Gamma_r(z_r)$, in particular $\Gamma_r(z_r) = U_r z_r$, which means that the resource provides its full nominal capacity U_r only if activated ($z_r = 1$), while offering zero capacity otherwise. For each resource $r \in \mathcal{R} \setminus \mathcal{Y}$, we impose $\Gamma_r(z_r) = U_r$, which means that the resource is already opened.

General Resource–Shared MILP Formulation

The formulations considered in this thesis can be written as follows.

$$\min \quad \sum_{e \in \mathcal{E}} \sum_{o \in \mathcal{O}_e} c_{e,o}\, x_{e,o} \;+\; \sum_{r \in \mathcal{Y}} F_r\, z_r \tag{5.1}$$

$$\text{s.t.} \quad \sum_{o \in \mathcal{O}_e} x_{e,o} \;=\; b_e \qquad \forall e \in \mathcal{E} \tag{5.2}$$

$$\sum_{e \in \mathcal{E}} \sum_{o \in \mathcal{O}_e} u_{e,o}(r)\, x_{e,o} \;\leq\; \Gamma_r(z_r) \qquad \forall r \in \mathcal{R} \tag{5.3}$$

$$x_{e,o} \in \mathcal{X} \qquad \forall e \in \mathcal{E},\ o \in \mathcal{O}_e \tag{5.4}$$

$$z_r \in \{0, 1\} \qquad \forall r \in \mathcal{Y} \tag{5.5}$$

where $\mathcal{X}$ encodes the domain of $x_{e,o}$ (*e.g.* $\mathbb{Z}_+$ or $\mathbb{R}_+$ with limits). In models without activation decisions, the second term of the objective function and the variables z_r are omitted, while $\Gamma_r(z_r) = U_r$. Although the framework theoretically accommodates further linear constraints (*e.g.*, side constraints or valid inequalities) without altering the decomposition logic, the case studies presented in this work map directly to the core structure defined by (5.1) – (5.5).

Subproblem Formulation and Merging Logic

This section operationalizes the decomposition logic by providing the concrete mathematical definition of the operator SolveCluster and the concatenation logic of the variables of MergeSolutions, as defined in Section 3.2.1.
Let $\mathcal{P} = \{C_1, \ldots, C_m\}$ be the partition produced by the constrained NJ procedure described in Section 4.2. The SolveCluster operator acts by restricting the global template (5.1)–(5.5) to the subset of entities C_k. Formally, for each cluster $k = 1, \ldots, m$, it solves the following subproblem:

$$\min \quad \sum_{e \in C_k} \sum_{o \in \mathcal{O}_e} c_{e,o}\, x_{e,o}^{(k)} \; + \; \sum_{r \in \mathcal{Y}_k} F_r\, z_r^{(k)} \tag{5.6}$$

$$\text{s.t.} \quad \sum_{o \in \mathcal{O}_e} x_{e,o}^{(k)} = b_e \qquad \forall e \in C_k \tag{5.7}$$

$$\sum_{e \in C_k} \sum_{o \in \mathcal{O}_e} u_{e,o}(r)\, x_{e,o}^{(k)} \le \Gamma_r\big(z_r^{(k)}\big) \qquad \forall r \in \mathcal{R}. \tag{5.8}$$

$$x_{e,o}^{(k)} \in \mathcal{X} \qquad \forall e \in C_k,\ o \in \mathcal{O}_e \tag{5.9}$$

$$z_r^{(k)} \in \{0,1\} \qquad \forall r \in \mathcal{Y}_k \tag{5.10}$$

Where

$$\mathcal{Y}_k \;:=\; \bigcup_{e \in C_k} \mathcal{O}_e \tag{5.11}$$

and it is not requested that $\mathcal{Y}_i \cap \mathcal{Y}_j = \emptyset$ for $i \neq j$. This yields a set of partial solutions $\{Sol_k\}_{k=1}^m$, defined by $\{x^{(k)}\}_{k=1}^m$ and possibly $\{z_r^{(k)}\}_{k=1}^m$. Subsequently, the operator MergeSolutions reconstructs a global draft solution from these partial solutions. Since the entity set is partitioned (*i.e.*, each $e \in \mathcal{E}$ belongs to exactly one cluster), the decision variables x are combined by simple concatenation:

$$x_{e,o}^{\text{merge}} := x_{e,o}^{(k)} \quad \forall k \in \{1, \ldots, m\}, \forall e \in C_k, \tag{5.12}$$

while activation variables (if present) are reconciled by a case–specific rule. In this thesis, we adopt the natural *union* rule:

$$z_r^{\text{merge}} \;:=\; \max_{k=1,\ldots,m} z_r^{(k)}, \qquad \forall r \in \mathcal{Y}_k \tag{5.13}$$

This choice ensures consistency: if any cluster uses resource r, then r is considered open in the merged solution; this completes the definition of the merged

solution Sol^{merge}. Because clusters are solved independently using the full nominal capacities (eq. (5.8)), the merged solution x^{merge} acts as a superposition of local plans and may violate the global resource capacities when evaluated jointly.

Feasibility Assessment

The first step after merging is to verify the global consistency of the solution $Sol^{merge} = (x^{\text{merge}}, z^{\text{merge}})$. We instantiate the abstract definition Load_e introduced in Section 3.2.1 to explicitly calculate the consumption of resources. The concrete load on each resource $r \in \mathcal{R}$ by entity e is calculated by aggregating the usage of all assigned options.

$$\text{load}_e(r) := \sum_{o \in \mathcal{O}_e} u_{e,o}(r), x_{e,o}^{\text{merge}} \tag{5.14}$$

Consequently, the global load is defined as follows:

$$\text{load}(r) := \sum_{e \in \mathcal{E}} \text{load}_e(r) \tag{5.15}$$

and finally, the concrete residual is defined as:

$$\text{res}(r) := \Gamma_r(z_r^{\text{merge}}) - \text{load}(r) \tag{5.16}$$

This allows us to formally define the $\mathsf{IsFeasible}$ operator. A solution is globally feasible with respect to capacities if and only if all residuals are non-negative:

$$\mathsf{IsFeasible}(Sol^{\text{merge}}, \mathcal{I}) = \text{True} \iff \text{res}(r) \geq 0 \quad \forall r \in \mathcal{R} \tag{5.17}$$

If condition (5.17) holds, the pipeline terminates, returning Sol^{merge}. Otherwise, the strategy proceeds to the repair phase described in the following.

Restoration Model Formulation

If infeasibility is detected, the $\mathsf{RepairAndReducedSolve}$ operator is activated. This phase requires decomposing the merged solution into a stable component and a modifiable component, followed by a re-optimization step.

Solution Decomposition. Assume that the iterative selection of candidates (the inner loop of Algorithm 2) has identified a set of decision variables to be modified. For each $e \in \mathcal{E}$ and $o \in \mathcal{O}_e$, let $\rho_{e,o}$ denote the amount of atomic units *released* from the merged assignment $x_{e,o}^{\text{merge}}$, such that $0 \leq \rho_{e,o} \leq x_{e,o}^{\text{merge}}$. The solution is thus decomposed into a *fixed part* x^{fixed}, acting as a rigid boundary condition, and a *released part* ρ available for re–optimization:

$$x_{e,o}^{fixed} := x_{e,o}^{\text{merge}} - \rho_{e,o} \tag{5.18}$$

We define the total released amount for entity e as $\rho_e := \sum_{o \in \mathcal{O}_e} \rho_{e,o}$, and the *repair set* $\mathcal{E}'$ as the subset of entities involved in the re-optimization:

$$\mathcal{E}' := e \in \mathcal{E} : \rho_e > 0 \tag{5.19}$$

Note that the specific logic for determining which $\rho_{e,o}$ are strictly positive (the Candidates, Score, and Release operators) is case–study dependent and detailed later.

Reduced MILP Formulation. The SolveReduced operator is generally instantiated as a variation of the global model (5.1)–(5.5), restricted to the released variables. We introduce the decision variables $\hat{x}_{e,o}$ for each $e \in \mathcal{E}'$ and $o \in \mathcal{O}_e$. The constraints are modified as follows:

1. **Demand Conservation:** Entities in the repair set must reassign exactly the amount that was released.

$$\sum_{o \in \mathcal{O}e} \hat{x}_{e,o} = \rho_e \qquad \forall e \in \mathcal{E}' \tag{5.20}$$

2. **Residual Capacity:** Capacity constraints are enforced on the *residual capacities* available after accounting for the fixed part of the solution.

$$\sum_{e \in \mathcal{E}'} \sum_{o \in \mathcal{O}_e} u_{e,o}(r)\, \hat{x}_{e,o} \;\le\; \Gamma_r(\hat{z}_r) \;-\; \sum_{e \in \mathcal{E}} \sum_{o \in \mathcal{O}_e} u_{e,o}(r)\, x^{fixed}_{e,o} \qquad \forall r \in \mathcal{R} \tag{5.21}$$

Depending on the specific application, the activation variables $\hat{z}$ can be fixed to z^{merge} or re-optimized together with $\hat{x}$. Solving this reduced model yields the optimal reallocation of the units released. The final reconstructed solution Sol^{final} is given by:

$$x^{final}_{e,o} := x^{fixed}_{e,o} + \mathbb{I}\left[e \in \mathcal{E}'\right] \hat{x}_{e,o} \tag{5.22}$$

$$z^{final} := \begin{cases} z^{merge}, & \text{if activation is fixed after merging} \\ \hat{z}, & \text{if activation is re-optimized} \end{cases} \tag{5.23}$$

where $\mathbb{I}[\cdot]$ is the indicator function.

5.2 The Space–Time Capacitated Path Assignment Problem

This case study models the allocation of discrete travel demand over a *space–time network* under *node capacity* constraints. The physical system is represented by a directed graph capturing feasible movements in space; congestion

and temporal interactions are introduced by expanding the network over a finite time horizon. For each OD pair, the demand is composed of multiple indivisible *atomic units* (agents), and each unit selects one path from a *precomputed candidate set*. Importantly, different units of the *same* OD pair may choose different paths, which means that OD demand is *split between candidates* , while each unit remains indivisible.

Space–Time Network. Let $G = (V, A)$ be the directed *spatial* graph (*e.g.*, a grid or a road network), where the traversing of an arc represents movement between spatial nodes. Given a discrete time horizon $\mathcal{T} = \{0, 1, \ldots, T\}$, we build the set of extended nodes in time $V^{\mathcal{T}} := V \times \mathcal{T}$. Each space–time node (v, t) represents the spatial node v in time t. A feasible path is then a sequence of space–time visits (and arcs) that starts at an origin node at $t = 0$ and reaches the destination within the horizon. Wait actions can be modeled as explicit *wait arcs* $(v, t) \to (v, t + 1)$. We ignore what happens during instants t and $t + 1$.

Capacity Coupling. Congestion is expressed by the capacities on the space–time nodes: at most $U_{(v,t)}$ units can occupy node v at time t. Since candidate paths may overlap in space and time, independently chosen routes can violate capacities, which is precisely the type of coupling addressed by the pipeline.

Sets and Parameters

We instantiate the generic template of Section 5.1 as follows:

- **Entities:** $\mathcal{E}$ is the set of OD pairs. Each entity $e \in \mathcal{E}$ is associated with an origin node $s_e \in V$, a destination node $d_e \in V$, and a demand $b_e \in \mathbb{Z}_+$ interpreted as the number of atomic units to travel from s_e to d_e.
- **Resources:** $\mathcal{R}$ is the set of space–time nodes, *i.e.* $\mathcal{R} = V^{\mathcal{T}} = \{(v, t) : v \in V,\ t \in \mathcal{T}\}$. Each resource $r = (v, t)$ has a capacity U_r, limiting the number of units that can occupy node v at time t.
- **Options (Candidate Paths):** for each entity $e \in \mathcal{E}$, $\mathcal{O}_e$ is a finite set of space–time paths defined by a list of resources (v, t), from $(s_e, 0)$ to $(d_e, \tau_{e,o})$, for some $\tau_{e,o} \leq T$. Candidate sets are generated offline (*e.g.* via k-shortest variants, bounded detours, or delay augmentation), and are treated as fixed input to the optimization model.
- **Costs:** $c_{e,o} \geq 0$ is the cost of assigning one unit of the OD pair e to the candidate path o. In our implementation, $c_{e,o}$ represents the *travel delay* between the length of the path o and the length of the shortest path from s_e to d_e.

Decision Variables

The decision variable $x_{e,o}$ counts how many atomic units of the OD pair e are assigned to the candidate path o. Since units are indivisible, the natural domain is:

$$x_{e,o} \in \mathbb{Z}_+ \qquad \forall e \in \mathcal{E},\ o \in \mathcal{O}_e.$$

In this case study, no activation/opening decisions are made (so $\mathcal{Y} = \emptyset$ and variables z are omitted).

Resource Usage

Resource usage encodes node occupancy in space–time. For a resource $r = (v,t) \in \mathcal{R}$ and a candidate path $o \in \mathcal{O}_e$, we define:

$$u_{e,o}(r) \ := \ \mathbb{I}\,[r \in o] \tag{5.24}$$

so that each assigned unit contributes one unit of load to every space–time node visited by its chosen path.

Global Formulation

Under the above identification, the generic MILP template (5.1)–(5.5) reduces to an integer path–assignment model:

$$\min \quad \sum_{e\in\mathcal{E}} \sum_{o\in\mathcal{O}_e} c_{e,o}\, x_{e,o} \tag{5.25}$$

$$\text{s.t.} \quad \sum_{o\in\mathcal{O}_e} x_{e,o} \ = \ b_e \qquad \forall e \in \mathcal{E} \tag{5.26}$$

$$\sum_{e\in\mathcal{E}} \sum_{o\in\mathcal{O}_e} u_{e,o}(v,t)\, x_{e,o} \ \le \ U_{(v,t)} \qquad \forall (v,t) \in V^{\mathcal{T}} \tag{5.27}$$

$$x_{e,o} \in \mathbb{Z}_+ \qquad \forall e \in \mathcal{E},\ o \in \mathcal{O}_e \tag{5.28}$$

The constraint (5.26) ensures that each OD pair routes exactly b_e atomic units, while (5.27) enforces the node capacities in space–time.

Feasibility Restoration

To complete the application of the strategy for this case study, we instantiate the three case–specific operators required by the feasibility restoration phase (Algorithm 2): Candidates, Score, and Release. Here, a violated resource is a *space–time node* $r^\star = (v^\star, t^\star) \in V^{\mathcal{T}}$ with a negative residual, *i.e.*, $\mathrm{res}(v^\star, t^\star) < 0$. Restoration is performed by iterative *release of one atomic unit at a time* from the fixed part of the merged solution; since units assigned to the same pair (e,o) are indistinguishable, each iteration operates on a candidate *bucket* (e,o) with $x^{\mathrm{fixed}}_{e,o} > 0$.

Candidate Selection For a critical resource $r^\star = (v^\star, t^\star)$, the set of candidates is defined as

$$\Phi_{(v^\star,t^\star)} := \left\{(e,o) \,:\, e \in \mathcal{E},\ o \in \mathcal{O}_e,\ x_{e,o}^{\text{fixed}} > 0,\ \ (v^\star,t^\star) \in o\right\} \tag{5.29}$$

That is, $\Phi_{(v^\star,t^\star)}$ contains exactly those assignments that (i) currently carry at least one atomic unit in the fixed solution and (ii) contribute to the load of the violated space–time node $(v^\star, t^\star)$. Releasing a candidate outside this set would not decrease the overload at $(v^\star, t^\star)$.

Scoring Strategy The scoring operator ranks candidates $(e,o) \in \Phi_{(v^\star,t^\star)}$ by estimating the existence of a "safe" alternative within the same OD candidate set. For each alternative $\tilde{o} \in \mathcal{O}_e \setminus \{o\}$, define its bottleneck residual as

$$B(\tilde{o}) := \min_{(v',t') \in \tilde{o}} \text{res}(v',t') \tag{5.30}$$

The score assigned to the candidate (e,o) is then

$$\sigma_{(e,o)} := \max_{\tilde{o} \in \mathcal{O}_e \setminus \{o\}} B(\tilde{o}) = \max_{\tilde{o} \in \mathcal{O}_e \setminus \{o\}} \ \min_{(v',t') \in \tilde{o}} \text{res}(v',t') \tag{5.31}$$

This max–min criterion prioritizes candidates for which there exists an alternative path whose *worst* space–time node still exhibits large residual slack, thus reducing the risk of propagating infeasibilities after reallocation.

Release Logic For this problem, the releases are made at the granularity of a single atomic unit. If candidate (e,o) is selected, the release operator removes one unit from the fixed assignment:

$$\rho_{e,o} \leftarrow \rho_{e,o} + 1, \qquad x_{e,o}^{\text{fixed}} \leftarrow x_{e,o}^{\text{fixed}} - 1 \tag{5.32}$$

As a consequence, the residual of every space–time node visited by o increases by one, since one occupant has been removed from each resource along the path. The procedure repeats until all critical resources satisfy the current tolerance threshold, at which point the released amounts ρ define the repair set $\mathcal{E}'$ and the reduced optimization is invoked as described in Section 5.1.

5.3 The Capacitated Facility Location Problem

This case study considers the *Capacitated Facility Location Problem* (CFLP) in two variants: a *single–source* formulation, in which each entity must be served by exactly one facility, and a *splittable–demand* formulation, in which an entity's demand may be fractionally distributed across multiple facilities. Both variants share the same underlying resource–sharing structure: facilities provide limited capacity, and entities compete for these capacities through assignment decisions. In this section, we map the general framework to this specific case study.

Sets and Parameters

We instantiate the generic template of Section 5.1 by identifying:

- **Entities:** $\mathcal{E}$ is the set of clients, each with a demand $b_e \geq 0$.
- **Resources:** $\mathcal{R}$ is the set of facilities, each with a capacity $U_r \geq 0$.
- **Assignment Costs:** $c_{e,r} \geq 0$ is the cost per unit of serving the client e from the facility r.
- **Activation:** we set $\mathcal{Y} = \mathcal{R}$.
- **Options:** for each client e, the admissible facilities are $\mathcal{O}_e \subseteq \mathcal{R}$ (*e.g.*, the closest k).

Decision Variables

The decision variables are those of the generic framework, specialized to assignments and openings:

- $x_{e,r} \geq 0$: amount of demand of entity e assigned to facility $r \in \mathcal{O}_e$. The domain depends on the variant;
- $z_r \in \{0, 1\}$: facility opening decision.

Resource usage follows directly from the identification of resources with facilities:

$$u_{e,r}(\tilde{r}) \;=\; \mathbb{I}[\tilde{r} = r], \qquad \forall e \in \mathcal{E},\; r \in \mathcal{O}_e,\; \tilde{r} \in \mathcal{R}, \tag{5.33}$$

Since the facilities are all different from each other, if we fix $\tilde{r} \in \mathcal{R}$, the left side of (5.3) could be written as:

$$\sum_{e \in \mathcal{E}} \sum_{r \in \mathcal{O}_e} u_{e,r}(\tilde{r}) x_{e,r} = \sum_{e \in \mathcal{E}} \sum_{r \in \mathcal{O}_e} \mathbb{I}[\tilde{r} = r] x_{e,r} = \sum_{e \in \mathcal{E}: \tilde{r} \in \mathcal{O}_e} x_{e,\tilde{r}} \tag{5.34}$$

5.3.1 Single–Source

In the single–source variant, each entity must be assigned to exactly one open facility. The model is as follows.

$$\min \quad \sum_{e \in \mathcal{E}} \sum_{r \in \mathcal{O}_e} c_{e,r}\, x_{e,r} \;+\; \sum_{r \in \mathcal{R}} F_r\, z_r \tag{5.35}$$

$$\text{s.t.} \quad \sum_{r \in \mathcal{O}_e} x_{e,r} \;=\; b_e \qquad \forall e \in \mathcal{E} \tag{5.36}$$

$$\sum_{e \in \mathcal{E}: r \in \mathcal{O}_e} x_{e,r} \;\leq\; U_r\, z_r \qquad \forall r \in \mathcal{R} \tag{5.37}$$

$$x_{e,r} \in \{0, b_e\} \qquad \forall e \in \mathcal{E},\; r \in \mathcal{O}_e \tag{5.38}$$

$$z_r \in \{0, 1\} \qquad \forall r \in \mathcal{Y} \tag{5.39}$$

The constraint (5.37) imposes the capacity of the facility and, in particular, forces $x_{e,r} = 0$ on all e whenever $z_r = 0$. The single-source nature is expressed by the domain (5.38): for each entity $e \in \mathcal{E}$, exactly one facility is assigned the full amount b_e, and all other assignments are zero.

Equivalent Binary Assignment Form. The model (5.35)–(5.39) can be equivalently rewritten by introducing binary assignment variables $y_{e,r} \in \{0,1\}$ and enforcing $x_{e,r} = b_e\, y_{e,r}$. Substituting $x_{e,r}$ yields the standard SS–CFLP formulation:

$$\min \quad \sum_{e\in\mathcal{E}} \sum_{r\in\mathcal{O}_e} (b_e c_{e,r})\, y_{e,r} \;+\; \sum_{r\in\mathcal{R}} F_r\, z_r \tag{5.40}$$

$$\text{s.t.} \quad \sum_{r\in\mathcal{O}_e} y_{e,r} = 1 \qquad \forall e \in \mathcal{E} \tag{5.41}$$

$$\sum_{e\in\mathcal{E}:r\in\mathcal{O}_e} b_e\, y_{e,r} \le U_r\, z_r \qquad \forall r \in \mathcal{R} \tag{5.42}$$

$$y_{e,r} \in \{0,1\} \qquad \forall e \in \mathcal{E},\ r \in \mathcal{O}_e \tag{5.43}$$

$$z_r \in \{0,1\} \qquad \forall r \in \mathcal{Y} \tag{5.44}$$

In the computational framework, we adopt this binary form for SS–CFLP, as it makes the single–source restriction explicit and avoids two–point domains for flow variables.

5.3.2 Splittable–Demand

In the splittable–demand variant, each entity's demand can be split across multiple open facilities. The model remains within the same template, but the domain of x becomes continuous:

$$\min \quad \sum_{e\in\mathcal{E}} \sum_{r\in\mathcal{O}_e} c_{e,r}\, x_{e,r} \;+\; \sum_{r\in\mathcal{R}} F_r\, z_r \tag{5.45}$$

$$\text{s.t.} \quad \sum_{r\in\mathcal{O}_e} x_{e,r} \;=\; b_e \qquad \forall e \in \mathcal{E} \tag{5.46}$$

$$\sum_{e\in\mathcal{E}:r\in\mathcal{O}_e} x_{e,r} \;\le\; U_r\, z_r \qquad \forall r \in \mathcal{R} \tag{5.47}$$

$$x_{e,r} \in \mathbb{R}_+ \qquad \forall e \in \mathcal{E},\ r \in \mathcal{O}_e \tag{5.48}$$

$$z_r \in \{0,1\} \qquad \forall r \in \mathcal{Y} \tag{5.49}$$

Compared with the model (5.35)–(5.39), the only structural change is (5.48); consequently, $x_{e,r}$ switches from an all–or-nothing allocation to a continuous split. This difference has algorithmic consequences for feasibility restoration and post–processing after the cluster merge, which are discussed in the following.

Feasibility Restoration

To complete the application of the strategy to the CFLP, we instantiate the three case–specific operators required by the feasibility restoration phase (Algorithm 2): Candidates, Score, and Release. These operators dictate which assignments are modified when a facility $r^\star$ is overloaded.

Candidate Selection For both single–source (SS) and splittable–demand (SD) variants, the repair process focuses on alleviating congestion at the most critical facility $r^\star$. The candidate set $\Phi_{r^\star}$ consists of all clients currently contributing to the load of that facility in the fixed solution:

$$\Phi_{r^\star} \;:=\; \left\{e \in \mathcal{E} : x^{\text{fixed}}_{e,r^\star} > 0\right\} \tag{5.50}$$

Only entities with active flow on $r^\star$ are considered for release, ensuring that the repair action directly addresses the detected violation.

Scoring Strategies The scoring operator $\mathsf{Score}(e, \mathcal{I})$ ranks candidates to determine which client should be removed (or reduced) from $r^\star$ first. We adopt different strategies for the two variants to reflect the rigidity of the single–source constraint versus the flexibility of splittable flows. Importantly, in both variants, we consider the opening of facilities that were closed.

- **Splittable–Demand Scoring.** In SD–CFLP, a client can shift small amounts of demand to other facilities. The score is defined as the aggregate residual capacity available in the client's neighborhood. For a candidate $e \in \Phi_{r^\star}$, the score is:

$$\sigma^{\text{SD}}_e \;=\; \sum_{r \in \mathcal{O}_e} \max\left(0; \text{Res}(r)\right). \tag{5.51}$$

  This rule prioritizes clients who have the most "slack" capacity available across their allowed facilities, minimizing the risk that moving flow will trigger new violations elsewhere.

- **Single–Source Scoring.** In the SS–CFLP, moving a client is an all–or-nothing operation. The score assesses the existence of a feasible alternative assignment. Let S^{feas}_e be the set of alternative facilities capable of accommodating the full demand b_e:

$$S^{\text{feas}}_e \;=\; \{r \in \mathcal{O}_e \setminus r^\star : \text{Res}(r) \geq b_e\} \tag{5.52}$$

  For a candidate $e \in \Phi_{r^\star}$, the score is:

$$\sigma^{\text{SS}}_e \;=\; |S^{\text{feas}}_e| \tag{5.53}$$

  This strategy prioritizes clients who can definitely relocate to a new facility.

Release Logic The operator Release determines the quantity δ to subtract from the fixed assignment $x^{\text{fixed}}_{e,r^\star}$ and add to the released variable $\rho_{e,r^\star}$. This step handles the transition from the generic "load" concept to specific domain constraints.

- **Single–Source Release.** Due to the integrality constraint $x_{e,r} \in \{0, b_e\}$, partial releases are invalid. If client e is selected, the entire demand is released; thus $\delta = b_e$, or equivalently, the client is released. Consequently, $x^{\text{fixed}}_{e,r^\star}$ becomes 0, and the full demand b_e must be reassigned by the reduced solver.

- **Splittable–Demand Release.** In SD–CFLP, since $x^{\text{fixed}}_{e,r^\star}$ could be fractional, we adopt a *fractional–first* strategy. Let $\bar{x} = x^{\text{fixed}}_{e,r^\star}$ be the current flow. The release amount δ is calculated as:

$$\delta = \begin{cases} \bar{x} - \lfloor \bar{x} \rfloor, & \text{if } \bar{x} \notin \mathbb{Z} \\ 1, & \text{if } \bar{x} \in \mathbb{Z} \end{cases} \tag{5.54}$$

  This logic ensures that if the current assignment has a fractional component (*e.g.* 4.5), only the decimal part (0.5) is released first. If the assignment is already an integer, the algorithm proceeds by peeling off one unit of demand at a time until the violation is resolved or the client is fully removed.

Chapter 6

Experimental Settings

This chapter describes the experimental methodology adopted to evaluate the proposed clustering strategy on the two problems described previously: the space–time capacitated path assignment problem and the capacitated facility location problem, in both single source and splittable–demand versions. The focus is deliberately on *how* instances are generated and *how* performance is assessed. Quantitative results are reported separately in Chapter 7.

6.1 Instance Generation

All instances are embedded in an undirected square grid graph $G = (V, E)$ of the side n (*i.e.*, $|V| = n^2$), where the nodes correspond to the grid coordinates $v = (i, j)$, and the edges connect via 4–neighborhood adjacencies; each edge has a unit weight. The distances used for path enumeration (ST–CPA) and proximity ranking (CFL) are induced by the shortest path lengths in G. This choice is motivated by the principle of spatial locality: selecting neighbors based on graph distance prioritizes relevant interactions and potential conflicts, which are far more likely to occur between proximal nodes than between arbitrarily or randomly chosen pairs. To scale the size of the the size of the instance while controlling interaction across spatial regions, we populate the grid using a *quadrant* mechanism. Given a *base grid side* n_0, we build a larger grid that can be partitioned into 2×2 blocks (4 quadrants) or 3×3 blocks (9 quadrants). In particular, for a target number of populated quadrants q:

- if $q \leq 4$, we use a 2×2 layout and set the grid side at $n = 2n_0$;
- if $q > 4$, we use a 3×3 layout and set the grid side at $n = 3n_0$.

Only the first q quadrants are populated with entities (OD pairs or clients and facilities); the remaining blocks are left empty; the quadrants are ordered from left to right and from top to bottom. The spatial interaction between adjacent quadrants is governed by the offset parameter off, which defines the boundary overlap or separation as follows:

- $\mathsf{off} < 0$: the quadrants are strictly separated by a buffer of $|\mathsf{off}|$ units (rows and columns of empty nodes), ensuring no direct adjacency between entities in different quadrants.
- $\mathsf{off} = 0$: adjacent quadrants share exactly one boundary (a single row and column), acting as a common interface.
- $\mathsf{off} > 0$: quadrants overlap by $\mathsf{off} + 1$ rows/columns, increasing the density of shared nodes and the degree of inter-quadrant interactions.

ST–CPA Instances

Entities and Atomic Units. For each $e \in \mathcal{E}$, we set $b_e = 5$. Entities are sampled uniformly at random within the corresponding quadrant, imposing $s_e \neq d_e$ and avoiding duplicate OD pairs within the same instance.

Resources and Capacities. We assign a uniform capacity of 5 to each node $v \in V$. To avoid trivial infeasibility at OD endpoints due to simultaneous departures/arrivals, the capacity assigned to a source or destination node is increased to match the total number of atomic units having that node as a source/destination (thus automatically "adjusting" when multiple OD pairs share an endpoint). Capacities are time-independent at the node level and apply uniformly across time indices in space–time constraints, thus for each resource $r \in \mathcal{R}$, $U_r = 5$.

Option Sets. We set $k = 10$. For each OD pair e, we enumerate the k shortest simple paths between s_e and d_e in G. Let the length (in visited nodes) of the k-th shortest path $p = (v_0 = s_e, v_1, \ldots, v_{\tau_e} = d_e)$ (*i.e.*, the longest) be τ_e; we use τ_e as the time horizon associated with entity e. To enrich the option set without changing the spatial route, we generate additional delayed variants. For each of the k shortest paths $p = (v_0 = s_e, v_1, \ldots, v_L = d_e)$, we create a family of paths obtained by inserting an initial waiting time at the source: for every integer $\tau \in [1, 2, \ldots, \tau_e - (L + 1)]$, we define

$$p^{(\tau)} = (\underbrace{s_e, \ldots, s_e}_{\tau \text{ times}}, v_0, v_1, \ldots, v_L) \tag{6.1}$$

i.e., we repeat s_e for τ time steps so that the total length of the created path $p^{(\tau)}$ does not exceed τ_e. The final option set $\mathcal{O}_e$ is the union of the k shortest paths and all their delayed variants. This construction yields options that differ only by departure time (and, therefore, space–time occupancy), which is essential for modeling congestion effects.

CFL Instances

Entities, Facilities, and Spatial Placement. In both CFL variants, entities are clients, while shared resources are facilities. For each populated quadrant, we generate the following:

- n_f facilities, and
- $n_c = 5n_f$ clients,

i.e., a fixed $5:1$ client–to-facility ratio. The positions of the clients and facilities are sampled uniformly at random on the grid nodes within the quadrant, enforcing unique positions within each set.

Demands, Capacities, and Opening Costs. All clients have a uniform demand of 20, while each facility has a capacity of 170 and a fixed opening cost of 1000. This yields a global load factor of approximately 0.59 (total demand over total capacity), which avoids trivial feasibility while still leaving enough slack to make the location decisions non-degenerate. The opening cost is set to balance fixed setup and demand–weighted transportation costs. In our grid instances, clients are usually served by a *nearby* open facility; hence, the marginal transportation cost associated with one client is typically a few hundred units (demand multiplied by the distance to the facility). Setting the opening cost to 1000 makes opening an additional facility roughly comparable to the transportation cost of serving a small handful of average clients. As a result, the model does not collapse to the extremes of either opening everything (fixed costs negligible) or opening as little as possible (fixed costs dominant), and the solver must genuinely trade off consolidation against proximity.

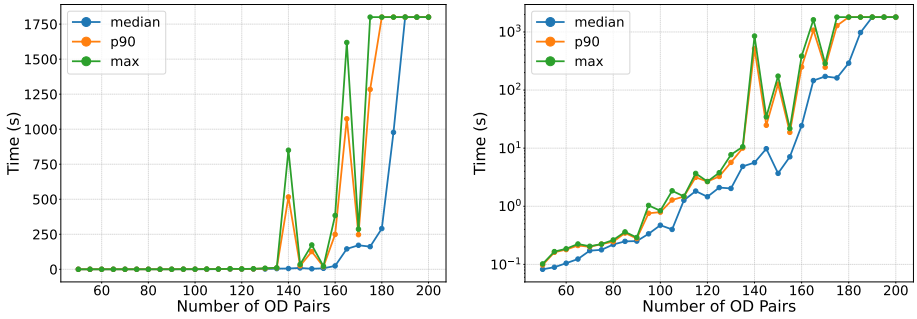
Option Sets. We also set $k = 10$ for CFL. For each client e, the admissible set $\mathcal{O}_e$ contains the k nearest facilities according to the shortest-path distance in G, both in the SS–CFL and SD–CFL variant.

6.2 Parameters Selection

A key design parameter of the strategy is the maximum admissible cluster size. We set this value through an *a priori* scalability study aimed at identifying the instance size at which global resolution begins to "blow up" in time. Let $\mathcal{E}_q$ be the set of entities per quadrant; the resulting threshold provides the baseline number of entities per quadrant $|\mathcal{E}_q|$, and consequently the maximum cluster size M. To determine this limit, we performed an incremental stress test by progressively increasing the size of the instance. For each size value, we generated 5 random instances (with different random seeds) to account for structural variability. Let $T_{\max} = 1800$s denote the time limit; for each size, we recorded the 5 resolution times (capped at $T_{\max}$ when a run hit the time limit) and summarized their distribution using robust statistics: the median, the 90th percentile (p90) and the maximum time (max). We then labeled an instance size as *failed* if it exhibited sustained difficulty according to one of the following criteria: (i) p90 $\geq 0.8 \times T_{\max}$, indicating that typical hard realizations are close to timing out; or (ii) at least 3 of 5 runs reached the time limit. The procedure was designed to end when 5 consecutive size increases were classified as failed, signaling that the computational breakdown point had been definitively reached.

ST–CPA Parameters Selection

For ST–CPA, we fix $n_0 = 20$, so the base grid is 20×20, and start from 50 OD pairs in a single quadrant; then we increase the number of OD pairs by steps of 5 (*i.e.*, $50, 55, 60, \dots$).



(a) Linear scale

(b) Logarithmic scale

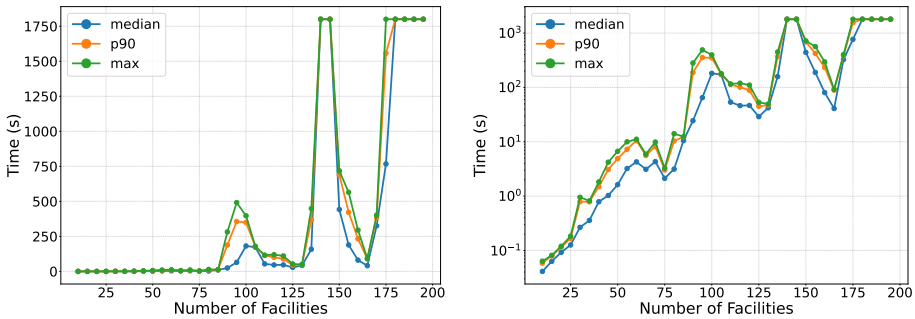
Figure 6.1: Scalability analysis for ST–CPA. The plots show the runtime distribution (median, p90, max) as a function of the number of OD pairs. The logarithmic scale highlights the exponential trend of the computational cost.

This study, based on Figure 6.1, supports the choice of a baseline number of 150 OD pairs per quadrant, so $|\mathcal{E}_q| = 150$, the point at which instabilities begin to appear. Since each OD pair generates 5 atomic units, $M = 5 \times |\mathcal{E}_q| = 750$ is the maximum cluster size in ST-CPA experiments.

CFL Parameters Selection

For CFL, we adopt the same methodology as used for ST–CPA: we conduct an incremental stress test on a 50×50 base grid ($n_0 = 50$), progressively increasing the problem size while maintaining a fixed $5:1$ client–to-facility ratio. As before, each size step is repeated on 5 random seeds, tracking the runtime distribution until the breakdown point is reached.

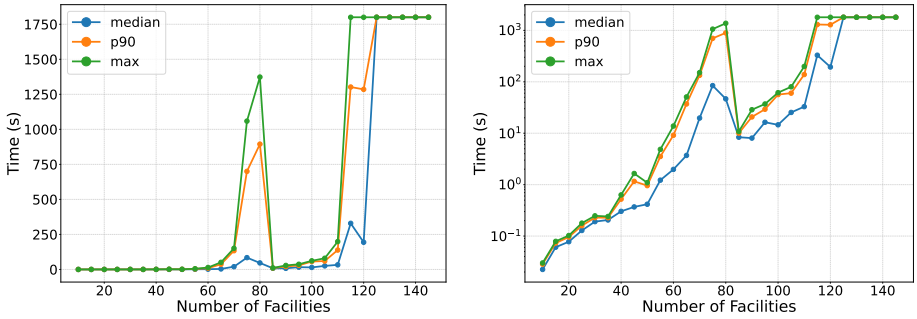
Single Source. We first analyze the Single Source variant. The evolution of computational time as a function of the number of facilities (and consequently, clients) is reported in Figure 6.2. These curves reveal a complex landscape, with early signs of volatility appearing around 90 facilities and a definitive computational breakdown occurring near 140 facilities. In light of this, we adopted a *conservative strategy* for the decomposition parameters. We selected a baseline size of 35 facilities, so $|\mathcal{E}_q| = 175$, an operating point located well within the stable region and significantly below the onset of instability. Since each client represents a single atomic entity with unit weight, $M = |\mathcal{E}_q| = 175$.



(a) Linear scale (b) Logarithmic scale

Figure 6.2: Scalability analysis for Single Source CFL.

Splittable Demand. We apply the same analysis to the Splittable-Demand variant, summarizing the runtime results in Figure 6.3.



(a) Linear scale (b) Logarithmic scale

Figure 6.3: Scalability analysis for Splittable Demand CFL.

The scalability profile shown in Figure 6.3 reveals the onset of computational instability around 70 facilities. As before, we adopt a *conservative strategy* to ensure reliable convergence, and we decided to select the same parameters, that is, a baseline size $|\mathcal{E}_q| = 175$ and $M = 175$.

6.3 Evaluation Protocol

To assess the performance of the proposed decomposition strategy, we designed an extensive experimental campaign focusing on scalability, solution quality, and computational efficiency. The primary goal is to demonstrate the method's ability to handle large–scale instances where global solvers may struggle, without significantly compromising the optimality of the solution. In this section, we first detail the construction of a scalable benchmark suite characterized by vary-

ing degrees of structural complexity and spatial interaction and subsequently describe the algorithmic configurations and metrics used for comparison. Each configuration (problem variant, number of quadrants, instance size, and offset) is repeated in 5 independent random seeds.

Instance Scaling

After selecting the baseline cluster sizes M, we generate larger instances by increasing the number of populated quadrants. For both ST-CPA and CFL, we consider

$$q \in \{2, 3, 4, 6, 7, 9\},$$

using the grid-side rules described in Section 6.1. Within each populated quadrant, we place the baseline number of entities $|\mathcal{E}_q|$ so that the global instance size $|\mathcal{E}|$ scales approximately linearly with q while preserving local density. To control the degree of interaction between quadrants, we vary the inter-quadrant offset off.
Specifically, for ST-CPA, we test

$$\mathsf{off} \in \{-1, 0, 1, 2, 5\},$$

while for CFL (both single-source and splittable-demand), we use

$$\mathsf{off} \in \{-15, -7, 0, 5, 10\}.$$

This difference is intentional: in ST-CPA, similarity is highly localized because two OD entities interact only when they share the *same node at the same time*. This means that even if two entities are close in the graph, they do not necessarily share many resources, so we do not need a large negative offset between quadrants to study the case of total separation between entities from different quadrants. In contrast, in CFL, two clients become similar as soon as they share *at least one candidate facility*. The CFL similarity signal is therefore less selective and would otherwise induce substantial cross–quadrant coupling even when quadrants are only mildly separated; larger offsets are used to probe regimes from weak to strong inter–quadrant interaction.

Compared Approaches

For each generated instance, we compute:

Global baseline. We solve the global MILP for the entire instance under a fixed time limit of 1800 seconds. When the baseline ends early, it provides an optimal solution, and when it hits the time limit, we use the best incumbent solution returned by the solver as the upper baseline bound. We record the solver status, the objective value (incumbent) and the runtime.

Heuristic pipeline. We run the proposed strategy with the selected maximum cluster size:

1. **Similarity and partitioning.** We compute the similarity matrix using the case study specific footprint overlap rule and partition entities via NJ to obtain clusters that respect the size constraint. We record the similarity–matrix and NJ computation times.
2. **Cluster solving.** Each cluster is solved as a standalone MILP. We record the total time taken to solve the clusters.
3. **Global feasibility check and repair.** If the union of cluster solutions violates global capacity constraints, we trigger the repair stage: we detect critical resources, release a minimal portion of the fixed solution, and solve a reduced global re-optimization that reallocates only the released part while keeping the rest fixed. We record the amount released and the runtime of the reduced resolution.

Performance Metrics

We assess performance along two axes: *solution quality* and *computational effort*.

Solution quality. Let z_{M} be the objective value returned by the global baseline (optimal or incumbent at the time limit), and let z_{H} be the final objective value returned by the heuristic pipeline (after repair, if triggered; otherwise, the sum of cluster objectives). We define the relative gap as follows:

$$gap^{\%} := 100 \times \frac{z_{\mathrm{H}} - z_{\mathrm{M}}}{z_{\mathrm{M}} + \epsilon}$$

Here, ϵ is a small value that avoids division by zero. For ST-CPA, z corresponds to the total delay accumulated by atomic units, whereas for CFL, z corresponds to opening costs plus shipping costs, consistent with the model definitions in Chapter 5.

Computational effort. We report total wall-clock time, and for the heuristic we also break it down into: (i) similarity-matrix computation + NJ partitioning, (ii) cluster model build + optimization, and (iii) repair (critical-resource identification + reduced global solve). We also report the number of clusters produced.

Computational Environment

All experiments have been run on an *Intel(R) Core(TM) i7-5930K CPU @ 3.50GHz* processor with 31 GB of RAM. The operating system used was *Ubuntu 22.04.5 LTS*. The entire framework was implemented in *Python 3.10.12*, and the MILP models were solved using *Gurobi Optimizer* version *12.0.3*, utilizing 12 threads with default parameter settings unless otherwise specified.

Chapter 7

Results

In this chapter, we present the experimental results obtained by applying the proposed strategy to the case studies defined in Chapter 5, specifically focusing on computational scalability and solution quality relative to a global baseline. These experiments were designed to answer the following *research questions*:

- **RQ1** How does the heuristic optimality gap (vs global baseline) evolve with the size of the problem across offsets?
- **RQ2** How does the heuristic runtime evolve with the size of the problem across offsets?
- **RQ3** What is the computational advantage of the heuristic over the global baseline?
- **RQ4** What is the impact of the repair phase on the overall performance of the heuristic across offsets?
- **RQ5** Is there a correlation between indicators and heuristic performances?

The evaluation is conducted on the synthetic datasets described in Chapter 6, covering a wide range of instance sizes and interaction densities. To ensure the robustness of the results against structural variability, every instance configuration is aggregated over 5 independent runs with different random seeds. Unless otherwise stated, the reported values represent the *median* across these 5 runs.

7.1 RQ1: Analysis of the Solution Quality

In the following sections, we analyze how the gap evolves in response to increasing offset and instance size for each problem class.

RQ1 on ST-CPA

We first focus on ST-CPA. Figure 7.1 illustrates the evolution of the optimality gap as a function of `offset`. The color gradient (from blue to yellow) represents the instance dimension (total number of ODs).

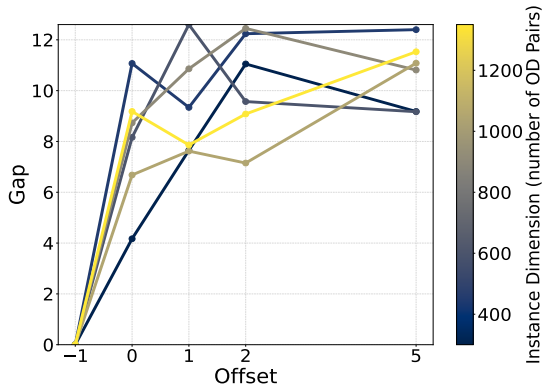


Figure 7.1: Optimality Gap as a function of the offset for ST-CPA instances. The color gradient indicates the instance dimension (number of OD pairs).

The visual analysis of Figure 7.1 reveals a clear trend. At `offset = -1` (isolated quadrants), the gap is negligible (near 0%), confirming that the decomposition introduces minimal overhead when sub-problems are independent. As the `offset` increases, introducing boundary interactions, we observe a natural increase in the gap. However, crucially, this growth does not explode; the gap remains controlled, mostly fluctuating between 5% and 15% between different instance sizes. The strategy demonstrates robustness, with the gap plateauing rather than linearly diverging with the offset.

RQ1 on SS-CFL

We extend the analysis to the SS-CFL problem. Figure 7.2 reports the trends of the optimality gap.
The results for SS-CFL highlight the effectiveness of the heuristic. For very negative offsets (offset $= -15$), the gap is practically zero or close to zero, indicating that the decomposition is exact when no boundaries are shared. Increasing the offset to positive values, the gap widens, but stabilizes immediately below 5%. In general, the heuristic is exceptionally robust for the single source variant of the facility location problem, producing solutions that are consistently within a 5% margin of optimality, regardless of the size of the instance or the degree of overlap.

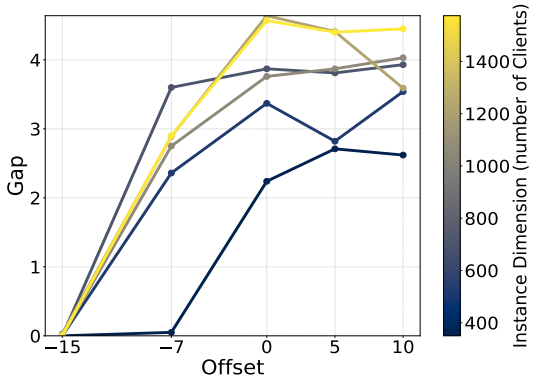


Figure 7.2: Optimality Gap as a function of the offset for ss-cfl instances. The color gradient indicates the instance dimension (number of clients).

RQ1 on SD-CFL

Finally, we analyze the SD-CFL problem in Figure 7.3.

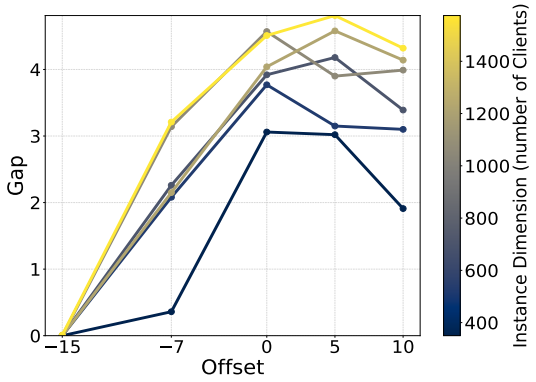


Figure 7.3: Optimality Gap as a function of the offset for SD-CFL instances. The color gradient indicates the instance dimension (number of clients).

The results for SD-CFL confirm the reliability of the strategy even when demand splitting is allowed; the gap increases almost linearly as the isolation between quadrants decreases (from offset -15 to 0), reaching a peak of approximately 5%. This mirrors the robust performance observed in the Single Source variant, proving that the decomposition strategy is effective regardless of the demand allocation type (splittable or non-splittable).

7.2 RQ2: Analysis of the Execution Time

To answer RQ2, we analyze the execution time of the strategy by measuring the total execution time (in seconds) required to return a solution. The goal is to assess how the computational effort scales when increasing both the instance dimension and the offset.

RQ2 on ST-CPA

We first analyze the ST-CPA problem. Figure 7.4 displays the trend of execution time.

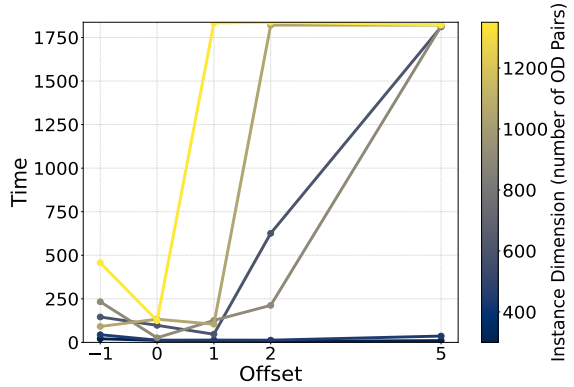


Figure 7.4: Execution Time as a function of the offset for ST-CPA instances. The color gradient indicates the instance dimension (number of OD pairs)

We observe a sharp phase transition. At `offset = -1`, the execution times are generally low (mostly below 250 s), even for larger instances. However, as soon as the offset increases to 0 or 1, the computational time for large instances increases, often hitting the 1800-second time limit. For offset 2 and 5 only small instances remain controlled in time, whereas large instances hit the time limit.

RQ2 on SS-CFL

We extend the computational analysis to the SS-CFL problem. Figures 7.5 report the execution times. It is immediately evident that the computational landscape for this problem is drastically different from the path assignment case. The time scale is orders of magnitude lower, with the maximum execution time never exceeding 13 seconds, even for the largest instances.
The execution time increases smoothly with the size of the instance, but remains negligible in absolute terms. We observe a slight peak around `offset = 0`, corresponding to the maximum ambiguity point for the boundary conditions, but the solver recovers quickly for positive offsets.

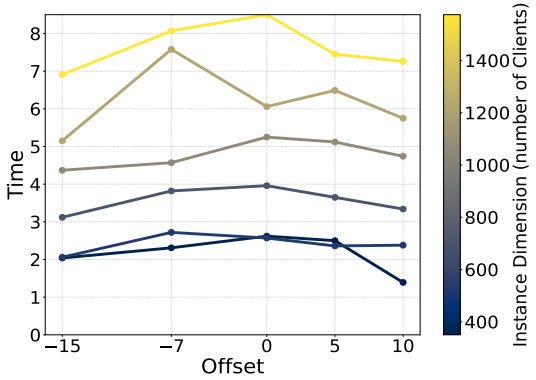


Figure 7.5: Execution Time as a function of the offset for SS-CFL instances. The color gradient indicates the instance dimension (number of clients)

RQ2 on SD-CFL

Finally, we examine the execution time of the SD-CFL problem. Figures 7.6 illustrate the execution times. The results are consistent with the Single Source case, demonstrating that the introduction of continuous flow variables for demand splitting does not negatively impact the solver's efficiency.

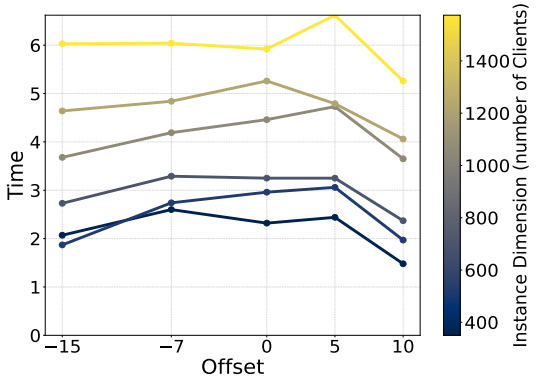


Figure 7.6: Execution Time as a function of the offset for SD-CFL instances. The color gradient indicates the instance dimension (number of clients)

The temporal profile of the SD-CFL follows a characteristic "bell shape" centered around the transition from isolation to interaction. The execution times are lowest at extreme negative offsets (where quadrants are isolated) and at large positive offsets. A noticeable peak occurs in the range of `offset` 0 to 5. This peak is driven by the maximum boundary coupling: intermediate offsets

create enough overlap to generate cross-cluster capacity conflicts, increasing the total time. For large positive offsets (*e.g.*, 10), the overlap becomes wide and, in these low-saturation instances, conflicts largely vanish, so the heuristic converges faster. However, the absolute values are extremely low: even the most complex instances are solved in less than 9 seconds. This reaffirms the strategy's capability to handle large-scale facility location problems with negligible computational cost.

7.3 RQ3: Computational Advantage

To address RQ3, we analyzed the execution times of the proposed heuristic compared to the global solver. Detailed tabular results for all configurations are provided in Appendix A.

RQ3 on ST-CPA

The results are detailed in Table A.1 and visualized in Figure 7.7, which plots the speedup (t_{glob}/t_{heur}) against the offset (off. The computational advantage of the heuristic exhibits a distinct "bell-shaped" behavior relative to the offset parameter, revealing three operational regimes. For $\mathsf{off} = -1$, the speedup is negligible, hovering around 1.0. In this scenario, the temporal constraints are sufficiently loose that the global solver can effectively exploit the problem's natural decomposability during its presolve phase, keeping t_{glob} low and rendering the heuristic's decomposition overhead unnecessary. In contrast, the heuristic delivers its maximum computational leverage at $\mathsf{off} \in \{0, 1\}$, where the global solver consistently hits the time limit ($TL = 1800$s). This regime corresponds to the apex of the bell curve, with speedups reaching up to 123.90× (for $\mathsf{q} = 3$), demonstrating the heuristic's ability to navigate complex temporal interdependencies that saturate the baseline. Finally, as the offset increases to 2 and 5, the speedup curves decrease. This degradation is directly attributable to the *Repair Phase* bottleneck, discussed later.

RQ3 on SS-CFLP

The computational performance for the Single Source variant is detailed in Table A.2 and visualized in Figure 7.8. In contrast to the ST-CPA results, the computational advantage here is primarily driven by the dimension of the instance rather than by the temporal offset. As illustrated by the separation of the curves in the figure, the heuristic's benefit becomes pronounced as the problem size increases. For smaller instances ($\mathsf{q} \in \{2, 3, 4\}$), the global solver remains efficient, often converging within seconds; consequently, the speedup is modest (typically $< 15\times$) and, in rare cases with minimal complexity ($\mathsf{q} = 2$), the heuristic matches rather than outperforms the baseline. However, a drastic divergence occurs for larger instances ($\mathsf{q} \geq 6$). In these scenarios, the global solver struggles significantly, frequently exhausting the available time ($TL = 1800$s)

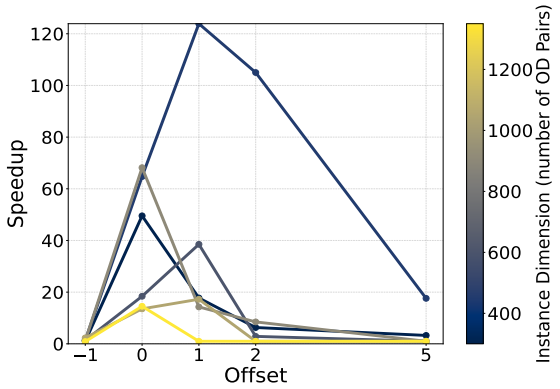


Figure 7.7: Speedup as a function of the Offset. The color gradient indicates the instance dimension (number of OD pairs)

across various offsets. In contrast, the heuristic demonstrates exceptional scalability, solving all instances in under 9 seconds, regardless of complexity. This performance gap yields massive speedups, systematically exceeding 200× for the largest instances ($\mathsf{q} = 9$), proving that the heuristic effectively breaks the computational barrier where the global solver becomes intractable.

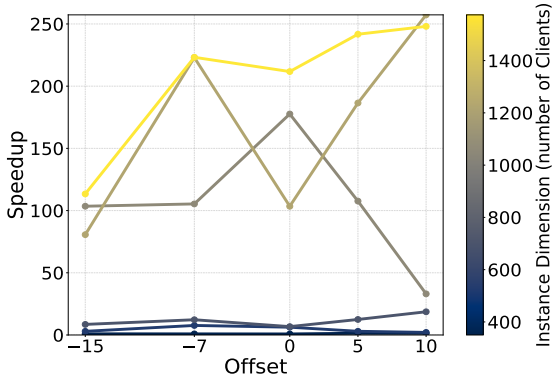


Figure 7.8: Speedup as a function of the Offset. The color gradient indicates the instance dimension (number of clients)

RQ3 on SD-CFLP

The performance analysis for the Splittable Demand variant is summarized in Table A.3 and is depicted in Figure 7.9. Similarly to the Single Source case, the computational advantage is primarily governed by the problem scale (q).

For small-scale instances ($\mathsf{q} \in \{2, 3\}$), the global solver is highly efficient, often outperforming the heuristic (resulting in speedups < 1.0) or yielding modest gains. However, as complexity increases, the superiority of the heuristic becomes overwhelming. For $\mathsf{q} \geq 6$, the global solver consistently hits the time limit in most offsets, whereas the heuristic solves even the largest instances in less than 7 seconds. This divergence leads to extreme speedups, peaking at 489.49× (for $\mathsf{q} = 6, \mathsf{off} = -15$). The trend confirms that the heuristic provides a crucial scalability benefit, effectively bypassing the computational bottleneck that the global solver encounters with high-dimensional demand splitting.

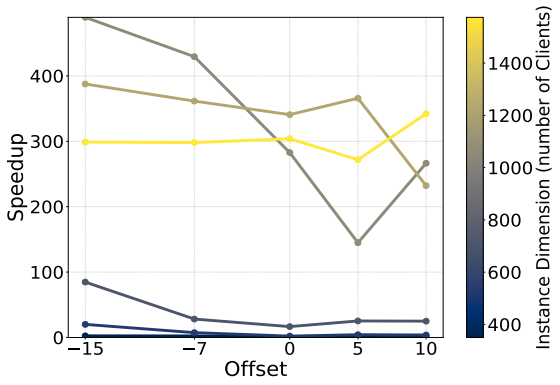


Figure 7.9: Speedup as a function of the Offset. The color gradient indicates the instance dimension (number of clients).

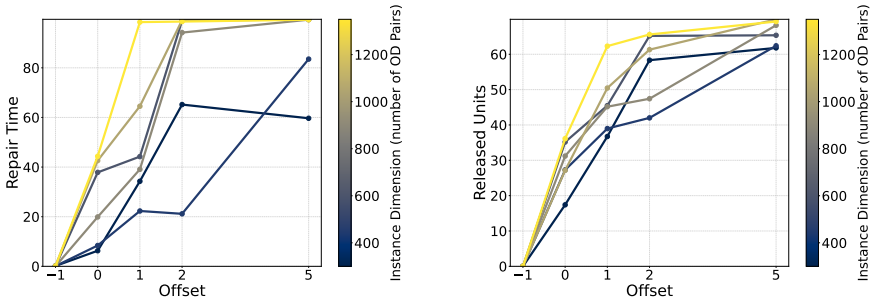
7.4 RQ4: Internal Dynamics

To answer RQ4, we examine the internal breakdown of the algorithm, specifically the *repair time percentage* ($t^{\%}_{rep}$) and the *percentage of re-optimized units* ($|\rho|^{\%}$), to quantify the cost of handling boundary conflicts. Detailed tabular results for all configurations are provided in Appendix A.

RQ4 on ST-CPA

The impact of the offset on the internal mechanics of the heuristic is visualized in Figure 7.10 and quantified in Table A.1. The data reveal a strictly monotonic relationship between the offset and the computational effort required for repair. For negative offsets ($\mathsf{off} = -1$), the problem structures are loosely coupled; consequently, the percentage of units released $|\rho|^{\%}$ is negligible (0.00%), and the repair time $t^{\%}_{rep}$ remains below 0.25%. This confirms that the initial decomposition is nearly optimal and requires minimal correction. However, as the offset becomes non-negative, the interaction between time steps intensifies,

forcing the algorithm to unfix a larger portion of the solution to restore feasibility. At $\mathsf{off} = 0$ and $\mathsf{off} = 1$, we observe a sharp transition: $|\rho|^{\%}$ jumps to approximately 30–50%, and the repair time occupies a significant portion of the total runtime ($t^{\%}_{rep}$ increasing from $\sim 6\%$ to $\sim 64\%$ depending on the size of the instance). The saturation point is reached at high offsets ($\mathsf{off} \geq 2$). Here, the dependencies are so pervasive that the repair phase dominates the entire execution. For $\mathsf{off} = 5$, the algorithm spends nearly all its time in the repair phase ($t^{\%}_{rep} > 99\%$) and re-optimizes up to 69% of the variables. This explains the performance degradation observed in *RQ*3: when the repair phase essentially re-solves the majority of the problem variables, the advantage of the decomposition strategy is nullified.



(a) Offset vs Repair Time

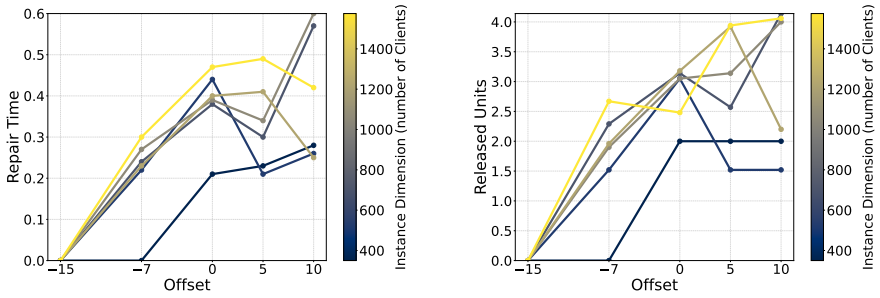
(b) Offset vs Released Units

Figure 7.10: Impact of the temporal offset on the internal dynamics of the heuristic for ST-CPA. Increasing the offset leads to higher coupling, resulting in a larger percentage of re-optimized units and increased repair time.

RQ4 on SS-CFLP

The internal dynamics of the heuristic applied to the SS-CFL problem, detailed in Table A.2 and Figure 7.11, presents a stark contrast to the path assignment problem. Here, the repair phase exerts a negligible impact on the overall computational efficiency, demonstrating the algorithm's robustness across all offset configurations. For the negative offset scenario ($\mathsf{off} = -15$), the decomposition is perfect: the algorithm requires zero repair time and re-optimizes no variables ($|\rho|^{\%} = 0.00$), confirming that the relaxed temporal constraints allow for independent spatial solutions. More importantly, this stability is maintained even as the offset increases. Unlike the ST-CPA case, where higher offsets caused a saturation of the repair phase, the SS-CFLP instances show a marginal response to tighter coupling. As visualized in the plots, the percentage of released units ($|\rho|^{\%}$) remains consistently below 5% in all configurations, and the time dedicated to the repair phase ($t^{\%}_{rep}$) never exceeds 0.60% of the total runtime. This behavior indicates that the spatial clustering strategy is highly effective for these low-saturation instances of facility location, generating initial solutions that are

nearly feasible even under inter-temporal constraints. Consequently, the massive speedups observed in *RQ*3 are justified by the fact that the heuristic spends virtually its entire execution time in the efficient, decomposed clustering phase, avoiding the bottleneck of iterative re-optimization.



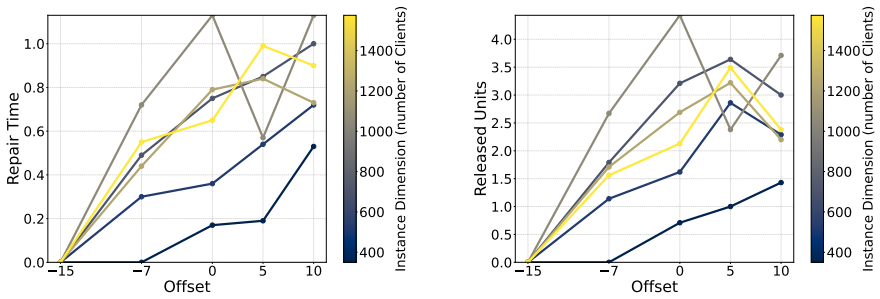
(a) Offset vs Repair Time

(b) Offset vs Released Units

Figure 7.11: Internal dynamics for SS-CFLP. In contrast to ST-CPA, the repair cost remains negligible ($< 0.6\%$) and the number of re-optimized units is minimal ($< 5\%$) regardless of the offset, indicating a highly stable decomposition.

RQ4 on SD-CFLP

The analysis of the Splittable Demand variant, presented in Table A.3 and Figure 7.12, confirms the robust behavior observed in the Single Source case. Despite the increased combinatorial freedom allowed by demand splitting, the heuristic maintains a highly stable internal structure where the impact of the repair phase is minimal. For loose temporal constraints ($\mathsf{off} \leq -7$), the algorithm achieves a near-perfect decomposition: the repair time is effectively zero, and the percentage of re-optimized units is negligible, indicating that the initial spatial clustering satisfies the temporal constraints without requiring adjustment. As the offset increases to $\mathsf{off} \geq 0$, we observe a slight physiological increase in repair metrics due to the tighter inter-temporal dependencies. However, these values remain well below the saturation levels seen in the ST-CPA problem. The maximum time spent in the repair phase is only 1.13% (for $\mathsf{q} = 6, \mathsf{off} = 0$), and the fraction of released units peaks at just 4.43%. This data demonstrates that the "cost" of handling splittable demand is absorbed almost entirely by the constructive clustering phase (which accounts for the majority of t_{heur}), while the iterative repair mechanism is required only for minor local adjustments. Thus, the efficiency of the heuristic is preserved across all offset configurations, preventing the computational explosion that affects the global solver.



(a) Offset vs Repair Time

(b) Offset vs Released Units

Figure 7.12: Internal dynamics for SD-CFLP. Similar to the SS-CFLP, the repair phase has a marginal impact, with repair times consistently below 1.2% and released units under 4.5%, confirming the stability of the approach even with demand splitting.

7.5 RQ5: Analysis of Presolve Indicators

In this final research question, we investigate the presolve indicators introduced in Chapter 3.3. The goal is to determine whether the structural properties measured *before* optimization (such as retention of similarity within clusters or expected congestion) correlate with the final performance of the heuristic.
To quantify this relationship, we computed the *Spearman rank correlation coefficient* between two sets of variables:

1. Similarity and congestion indicators.
2. The relative optimality gap ($gap\%$) and the total execution time of the heuristic (t_{heur}).

The Spearman coefficient was chosen because it detects monotonic relationships without assuming linearity. For the analysis of $gap\%$, we considered only instances where a valid optimal baseline was available to ensure the reliability of the metric, while for t_{heur} the entire data set was used.

RQ5 on ST-CPA

We start by analyzing the space-time-capacitated path assignment problem. The Spearman correlation coefficients are reported in Table A.4. From a purely statistical perspective, the data suggest a significant relationship between the clustering structure and the quality of the solution. In particular, the Similarity Index (R_{sim}) exhibits a strong negative correlation of -0.77 with the optimality gap. This coefficient indicates a clear monotonic trend: as the partition captures more similarity mass internally, the relative gap tends to decrease.
However, relying solely on rank correlation can be deceptive regarding the predictive precision of the indicator. To investigate the nature of this relationship,

we visualize the data in Figure 7.13, which plots the Similarity Index R_{sim} against the optimality gap ($gap\%$) for all instances where an optimal baseline was found. Each point is colored by the speedup $\frac{t_{glob}}{t_{heur}}$ and the size of the point is relative to the total time of the heuristic.

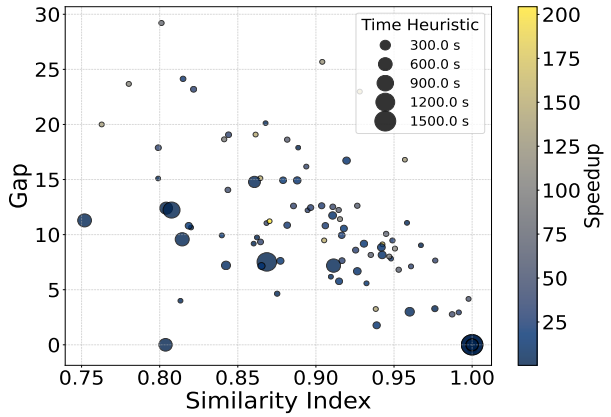


Figure 7.13: Scatter plot of Global Similarity Index (R_{sim}) vs. Optimality Gap ($gap\%$) for ST-CPA instances. The color gradient represents the speedup and the size represents the total heuristic time. While Spearman's coefficient suggests a correlation, the visual trend is noisy.

The scatter plot reveals a more nuanced reality than the single correlation coefficient suggests. Although the cluster of points in the lower-right region confirms that very high retention of similarity ($R_{\text{sim}} \gtrsim 0.98$) is consistently associated with near-zero gaps, the behavior in the intermediate range is highly variable. Specifically, for similarity values between 0.75 and 0.9, the resulting gap spans a wide interval, indicating that R_{sim} alone is not sufficient to precisely predict the magnitude of the optimality loss. The high Spearman value captures the global rank order, but the scatter plot highlights that significant structural noise remains. Regarding execution time (t_{heur}), Table A.4 shows moderate to strong positive correlations with congestion indicators, most notably E_{abs} (0.67) and E_{cross} (0.51). This is consistent with the findings of RQ4: strictly capacity-based metrics are better suited to anticipate the intensity of the repair phase (and thus the runtime) than similarity-based metrics.

RQ5 on SS-CFLP

The analysis of the single source facility location problem is presented in Table A.5 and visualized in Figure 7.14. Here, the predictive power of the presolve indicators with respect to the quality of the solution is remarkably strong. The Global Similarity Index (R_{sim}) shows a very high negative correlation (-0.87) with the optimality gap. Unlike the previous case study, the scatter plot con-

firms that this statistical figure corresponds to a clear and consistent visual trend: as R_{sim} increases towards 1.0, the gap decreases monotonically with minimal variance. This suggests that in facility location problems, where interaction is defined by sharing candidate facilities, the ability of the clustering to internalize these shared options is the primary determinant of optimality. When R_{sim} is high, the decomposition is structurally sound and the resulting independent subproblems yield a solution extremely close to the global optimum.

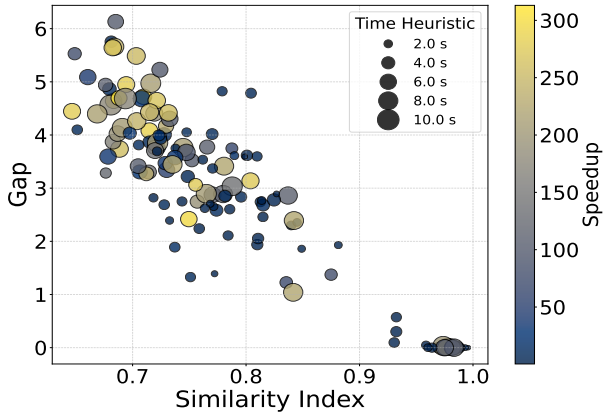


Figure 7.14: Scatter plot of Global Similarity Index (R_{sim}) vs. Optimality Gap ($gap\%$) for SS-CFL instances. The color gradient represents the speedup (t_{glob}/t_{heur}) and point size represents heuristic runtime. The trend here is much cleaner than in ST-CPA.

However, in terms of execution time (t_{heur}), the correlations are negligible (mainly < 0.2). This absence of correlation is explained by the structural properties of these instances. As discussed in RQ4, the generated SS-CFL instances are characterized by low saturation levels. Consequently, the repair phase is almost never triggered or is instantaneous regardless of the cluster configuration. In this context, congestion indicators (E_{abs}, etc.) become irrelevant to predict runtime because the computational cost is dominated by the constant overhead of solving independent clusters, rather than by a variable repair effort. Ideally, indicators successfully predict "how good" the solution will be (R_{sim}), but "how long" it will take is effectively constant and fast.

RQ5 on SD-CFLP

Finally, Table A.6 reports the results for the splittable demand variant. The pattern closely mirrors the single source case, confirming that the flexibility of demand splitting does not weaken the relationship between the presolve indicators and the final solution quality. We observe a very strong negative correlation between R_{sim} and the gap (-0.84). The scatter plot in Figure 7.15 confirms this statistic, displaying a well-defined trend where instances with high retention of

similarity consistently produce low optimality gaps. This reinforces the conclusion that R_{sim} is a robust proxy for the quality of decomposition between different variants of the facility location: as long as the partitioning respects the proximity preferences of the clients, the resulting subproblems provide a high-fidelity approximation of the global problem.

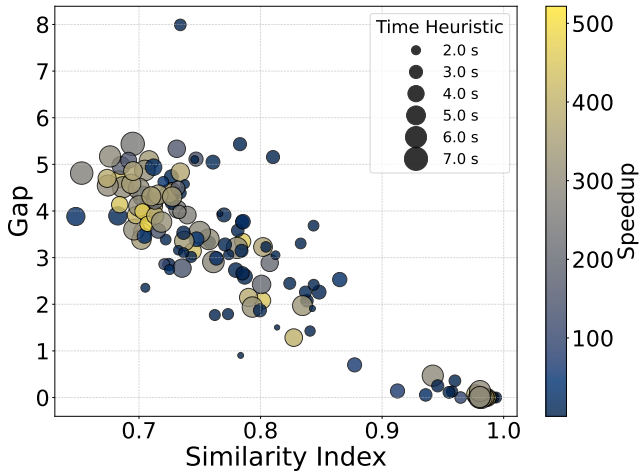


Figure 7.15: Scatter plot of Global Similarity Index (R_{sim}) vs. Optimality Gap ($gap\%$) for SD-CFL instances. Points are colored by speedup and sized by heuristic time. The clear trend confirms R_{sim} as a strong predictor of quality.

Regarding execution time (t_{heur}), correlations with congestion indicators remain weak (all below 0.15); as in the SS-CFL case, this is attributable to the low saturation of the generated instances. Because facility capacities are rarely bound to the point of forcing complex rerouting, the repair phase is minimal.

Chapter 8

Conclusions and Future Work

8.1 Conclusions

In this thesis, we addressed the scalability challenges of large-scale capacitated assignment problems modeled through Mixed-Integer Linear Programming. We proposed and evaluated a *clustering–based strategy* designed to improve computational tractability while maintaining controlled solution quality. The core premise of our work was that, under shared capacity constraints, entity interactions are often heterogeneous: strong interactions can be handled locally within bounded-size subproblems, while weaker interactions can be managed through a limited global coordination step. Accordingly, we developed a pipeline that partitioned the instance into size-constrained clusters using a Neighbor Joining algorithm, solved each cluster as a smaller independent MILP, merged the partial solutions, and restored feasibility through a targeted reduced global reoptimization when cross-cluster capacity violations occurred.

We assessed the methodology in two distinct case studies: the Space-Time Capacitated Path Assignment problem and the Capacitated Facility Location problem, considering both single-source and splittable-demand variants. We conducted an extensive experimental campaign on synthetic grid instances, scaling both the size (via the number of populated quadrants) and the interaction intensity (via the offset parameter).

The results consistently showed that the proposed strategy was most effective when cross-cluster coupling was moderate or localized, and that its behavior depended strongly on the structural properties of the underlying problem family. In the case of ST-CPA, we observed a clear trade-off between decomposition benefits and cross-cluster conflicts. In isolated regimes (negative offsets), the approach proved essentially exact, achieving negligible optimality gaps. As the interaction increased, the gap grew but remained controlled. However, the runtime exhibited a marked phase transition: large instances that were tractable un-

der separation rapidly became time-limited as boundary interactions increased. We found that in the most constrained regimes, characterized by high offsets or local saturation, the reduced global re-optimization dominated the computation, effectively turning the method into a large-neighborhood search that prioritized feasibility at the expense of speedup.
In contrast, for both SS-CFL and SD-CFL, we observed that the clustering strategy was intrinsically robust. We demonstrated that solution quality remained tightly controlled across all tested offsets and sizes, with execution times staying on the order of seconds even for the largest instances. Furthermore, the repair component was consistently negligible, indicating that the initial clustering phase successfully identified high-quality subsystems that required minimal post-processing. These results confirmed that the facility-location instances exhibited a strong exploitable local structure in the generated process tested.
Additionally, we investigated the relationship between structural presolve indicators and the heuristic's performance. We established that the Global Similarity Index (R_{sim}) served as a reliable predictor of solution quality (gap), particularly for facility location problems where a strong negative correlation was observed. In contrast, for the path assignment problem, we found that congestion-based metrics (E_{abs}) were more indicative of the computational effort required by the repair phase.
Taken together, empirical evidence supported the central conclusion of the thesis: a similarity-driven, size-constrained clustering strategy can make large capacitated assignment instances tractable for MILP-based resolution, provided that the underlying interaction structure is sufficiently local. When coupling becomes pervasive, the computational bottleneck shifts from subproblem resolution to global feasibility restoration.

8.2 Future Work

Building on the findings of this thesis, several directions will characterize future research to improve both the effectiveness and generality of the framework.
A first line of work will be on the similarity function used to drive clustering. Although the current design relied on overlap information, future research will introduce weighted similarities that emphasize scarce resources (*e.g.*, weighting overlaps by capacity tightness or expected congestion), incorporate cost sensitivity, or combine multiple signals into a composite measure. Complementarily, we will attempt to learn similarity weights from empirical feedback, using past instances to predict which pairs of entities are most likely to generate costly cross-cluster conflicts.
The second direction will focus on the partitioning strategy. We will compare the current NJ-based hierarchical backbone against alternative methods, such as constrained graph partitioning, multilevel coarsening, or community-detection approaches, to quantify how different topological structures impact the cost of the repair phase. The goal will be to determine whether partitions with smaller boundary interfaces yield a better balance between cluster cohesion and

separability than those based purely on pairwise similarity.

A third area will involve the definition of more accurate predictive indicators. Although we successfully correlated similarity retention with solution quality, predicting the runtime for complex path-assignment instances remained challenging. Future work will seek to define composite indicators capable of forecasting the "hardness" of an instance (specifically, the expected repair overhead) before optimization begins.

Finally, we will extend the framework to additional application domains that fit the pattern of entity-level decisions under shared capacities. We will apply the strategy to other network assignment models (*e.g.*, multi-commodity variants, time-dependent routing settings) and scheduling problems. Broadening the testbed will help characterize which structural features make a problem particularly amenable to clustering-based decomposition.

Appendix A

Detailed Experimental Results

This appendix reports the complete tabular results for all the experimental campaigns discussed in Chapter 7. The results are organized by problem type and further grouped by the offset parameter. The problem–specific constants defined in Chapter 6 (such as the base grid size and the maximum cluster size) remain fixed, while the instance size scales via the number of entities and populated quadrants. For each experimental configuration, we report the following metrics:

- $|\mathcal{E}_q|$: the total number of entities per quadrant.
- q: the number of populated quadrants.
- t_{glob}: the median runtime (in seconds) of the global solver.
- TL: the absolute count of runs (out of 5) that reached the time limit of 1800 s, reported separately for the global solver and the heuristic.
- t_{heur}: the median runtime (in seconds) of the proposed heuristic.
- $t^{\%}_{rep}$: the median percentage of the total heuristic time spent specifically in the repair phase.
- $t^{\%}_{clust}$: the median percentage of the time spent in the creation of the similarity matrix $t^{\%}_{sim}$ + the time spent in the neighbor joining phase $t^{\%}_{nj}$.
- $|\rho|^{\%}$: the median percentage of atomic units released and re-optimized during the repair phase, relative to the total demand.
- $|\mathcal{P}|$: the median number of clusters generated by the partitioning phase.
- $gap^{\%}$: the median relative gap between the heuristic objective value (z_H) and the global baseline (z_M).

- $\frac{t_{glob}}{t_{heur}}$: the median of the ratio between global and heuristic runtime.

A.1 Space-Time Capacitated Path Assignment

In this section, we present the results for the ST-CPA problem.

$\mathcal{I}$			**Global**		**Heuristic**						**Comparison**	
$\lvert\mathcal{E}_q\rvert$	q	off	t_{glob}	TL	t_{heur}	TL	$t^{\%}_{clust}$	$t^{\%}_{rep}$	$\lvert\rho\rvert^{\%}$	$\lvert\mathcal{P}\rvert$	$gap^{\%}$	$\frac{t_{glob}}{t_{heur}}$
150	2	−1	34.84	0	20.55	0	3.58	0.16	0.00	2	0.00	1.01
150	3	−1	74.07	0	44.08	0	2.01	0.11	0.00	3	0.00	1.40
150	4	−1	112.68	0	145.66	0	0.73	0.04	0.00	4	0.00	1.32
150	6	−1	350.81	1	233.11	0	0.77	0.04	0.00	6	0.00	1.27
150	7	−1	163.54	1	91.19	0	2.47	0.25	0.00	7	0.00	2.30
150	9	−1	355.13	0	457.09	0	0.68	0.03	0.00	9	0.00	0.88
150	2	0	634.46	2	11.62	0	10.38	6.28	17.40	3	4.17	49.54
150	3	0	795.79	2	12.64	0	13.50	8.40	27.29	4	11.07	64.73
150	4	0	1800.86	5	97.88	0	2.27	37.86	35.17	5	8.16	18.40
150	6	0	1801.24	5	26.42	0	7.88	19.90	31.20	7	8.73	68.18
150	7	0	1801.46	5	133.16	1	3.10	42.62	27.16	8	6.68	13.53
150	9	0	1802.18	5	124.45	2	4.25	44.36	36.13	11	9.18	14.53
150	2	1	211.94	0	9.44	0	8.03	34.28	36.73	3	7.65	17.74
150	3	1	1800.60	4	13.29	0	12.80	22.30	38.98	4	9.34	123.90
150	4	1	1800.90	5	46.81	0	4.58	44.22	45.47	5	12.60	38.51
150	6	1	1801.32	5	126.14	1	2.19	39.08	45.16	7	10.86	14.28
150	7	1	1801.64	5	104.44	1	2.59	64.52	50.44	8	7.62	17.25
150	9	1	1801.98	5	1832.73	4	0.22	98.43	62.30	11	7.86	0.98
150	2	2	41.65	0	5.70	0	16.53	65.19	58.33	3	11.05	6.31
150	3	2	1800.65	4	12.76	0	11.23	21.16	42.00	4	12.24	105.00
150	4	2	1800.91	5	625.88	2	0.22	99.03	65.20	5	9.57	2.88
150	6	2	1801.35	5	212.48	0	1.64	94.20	47.42	8	12.45	8.48
150	7	2	1801.60	5	1820.71	4	0.23	99.21	61.30	9	7.15	0.99
150	9	2	1802.27	5	1837.34	5	0.28	98.64	65.60	11	9.08	0.98
150	2	5	23.33	0	10.18	0	23.90	59.67	61.80	3	9.18	3.25
150	3	5	448.96	1	36.26	0	4.12	83.52	62.40	4	12.40	17.60
150	4	5	1800.90	5	1810.80	3	0.16	99.49	65.37	5	9.17	0.99
150	6	5	1801.55	5	1815.06	3	0.20	99.40	68.20	7	10.81	0.99
150	7	5	1802.02	5	1820.75	4	0.16	99.55	69.89	9	11.08	0.99
150	9	5	1802.09	5	1821.82	5	0.27	99.39	69.26	12	11.53	0.99

Table A.1: Comparison of Global vs. Heuristic approach on ST-CPA instances with $T_{limit} = 1800$s.

A.2 Single Source Capacitated Facility Location

This section details the computational performance on the Single Source Facility Location instances.

$\mathcal{I}$			**Global**		**Heuristic**						**Comparison**	
$\lvert\mathcal{E}_q\rvert$	q	off	t_{glob}	TL	t_{heur}	TL	$t^{\%}_{clust}$	$t^{\%}_{rep}$	$\lvert\rho\rvert^{\%}$	$\lvert\mathcal{P}\rvert$	$gap^{\%}$	$\frac{t_{glob}}{t_{heur}}$
175	2	−15	1.82	0	2.04	0	39.79	0.00	0.00	2	0.00	0.94
175	3	−15	5.60	0	2.06	0	35.99	0.00	0.00	3	0.00	2.95
175	4	−15	26.58	0	3.12	0	32.46	0.00	0.00	4	0.00	8.51
175	6	−15	478.21	0	4.37	0	30.13	0.00	0.00	6	0.00	103.50
175	7	−15	454.94	0	5.15	0	30.61	0.00	0.00	7	0.03	80.65
175	9	−15	876.98	1	6.91	0	29.58	0.00	0.00	9	0.00	113.40
175	2	−7	2.40	0	2.31	0	46.40	0.00	0.00	2	0.05	0.88
175	3	−7	20.86	0	2.72	0	30.47	0.22	1.52	4	2.36	7.67
175	4	−7	47.03	0	3.82	0	35.25	0.24	2.29	5	3.60	12.26
175	6	−7	567.15	0	4.57	0	29.11	0.27	1.90	7	2.75	105.35
175	7	−7	1800.28	3	7.58	0	24.89	0.23	1.96	9	2.89	222.98
175	9	−7	1800.35	4	8.07	0	28.73	0.30	2.67	11	2.90	223.20
175	2	0	2.17	0	2.62	0	43.74	0.21	2.00	3	2.24	0.83
175	3	0	18.00	0	2.57	0	34.10	0.44	3.05	4	3.37	6.27
175	4	0	26.56	0	3.96	0	35.41	0.38	3.14	5	3.87	6.73
175	6	0	959.72	1	5.25	0	27.77	0.39	3.05	8	3.76	177.63
175	7	0	642.03	1	6.06	0	32.08	0.40	3.18	10	4.64	103.46
175	9	0	1800.35	5	8.50	0	27.73	0.47	2.48	12	4.57	211.74
175	2	5	2.41	0	2.50	0	47.63	0.23	2.00	3	2.71	1.40
175	3	5	6.39	0	2.36	0	37.99	0.21	1.52	4	2.82	2.98
175	4	5	37.75	0	3.65	0	39.96	0.30	2.57	5	3.81	12.48
175	6	5	574.21	0	5.12	0	27.88	0.34	3.14	8	3.87	107.61
175	7	5	1195.22	1	6.49	0	31.28	0.41	3.92	10	4.41	186.48
175	9	5	1800.36	5	7.45	0	32.48	0.49	3.94	13	4.40	241.74
175	2	10	1.92	0	1.39	0	46.00	0.28	2.00	3	2.62	1.37
175	3	10	4.95	0	2.38	0	31.82	0.26	1.52	4	3.54	2.08
175	4	10	65.23	0	3.34	0	27.32	0.57	4.14	5	3.93	18.66
175	6	10	171.25	0	4.74	0	26.33	0.60	4.00	8	4.03	33.06
175	7	10	1212.04	2	5.75	0	28.84	0.25	2.20	9	3.59	257.31
175	9	10	1800.37	5	7.26	0	27.60	0.42	4.06	12	4.45	248.04

Table A.2: Comparison of Global vs. Heuristic approach on SS–CFLP instances with $T_{limit} = 1800$s.

A.3 Splittable-Demand Capacitated Facility Location

This section details the computational performance on the Splittable-Demand Facility Location instances.

$\mathcal{I}$			**Global**		**Heuristic**						**Comparison**	
$\lvert\mathcal{E}_q\rvert$	q	off	t_{glob}	TL	t_{heur}	TL	$t^{\%}_{clust}$	$t^{\%}_{rep}$	$\lvert\rho\rvert^{\%}$	$\lvert\mathcal{P}\rvert$	$gap^{\%}$	$\frac{t_{glob}}{t_{heur}}$
175	2	−15	5.20	0	2.07	0	75.81	0.00	0.00	2	0.00	2.51
175	3	−15	41.72	0	1.87	0	55.65	0.00	0.00	3	0.00	20.03
175	4	−15	231.11	0	2.73	0	60.32	0.00	0.00	4	0.00	84.71
175	6	−15	1800.07	5	3.68	0	58.54	0.00	0.00	6	0.00	489.49
175	7	−15	1800.09	5	4.64	0	61.65	0.00	0.00	7	0.00	387.58
175	9	−15	1800.12	5	6.03	0	61.34	0.00	0.00	9	0.01	298.76
175	2	−7	5.66	0	2.60	0	81.40	0.00	0.00	2	0.36	2.34
175	3	−7	19.90	0	2.74	0	73.65	0.30	1.14	4	2.08	7.39
175	4	−7	94.05	0	3.29	0	71.67	0.49	1.79	5	2.26	28.15
175	6	−7	1800.08	4	4.19	0	59.14	0.72	2.67	8	3.14	429.35
175	7	−7	1800.20	4	4.84	0	61.92	0.44	1.71	8	2.15	361.44
175	9	−7	1800.12	5	6.04	0	55.91	0.55	1.56	12	3.21	298.00
175	2	0	1.43	0	2.32	0	81.07	0.17	0.71	3	3.06	0.64
175	3	0	5.84	0	2.96	0	73.92	0.36	1.62	4	3.77	2.08
175	4	0	53.52	0	3.25	0	65.47	0.75	3.21	5	3.92	16.64
175	6	0	1215.37	1	4.46	0	60.89	1.13	4.43	8	4.57	282.65
175	7	0	1800.09	4	5.26	0	54.68	0.79	2.69	10	4.04	340.72
175	9	0	1800.13	5	5.92	0	54.10	0.65	2.13	12	4.51	303.89
175	2	5	1.01	0	2.44	0	78.51	0.19	1.00	3	3.02	0.41
175	3	5	12.98	0	3.06	0	72.55	0.54	2.86	4	3.15	4.24
175	4	5	78.34	0	3.25	0	66.32	0.85	3.64	5	4.18	25.25
175	6	5	690.89	1	4.73	0	58.59	0.57	2.38	9	3.90	144.85
175	7	5	1800.20	4	4.79	0	57.53	0.84	3.22	10	4.58	365.82
175	9	5	1800.11	5	6.62	0	48.09	0.99	3.49	12	4.81	271.82
175	2	10	1.06	0	1.48	0	69.93	0.53	1.43	3	1.91	0.60
175	3	10	7.35	0	1.97	0	56.15	0.72	2.29	4	3.10	3.81
175	4	10	55.31	0	2.37	0	52.30	1.00	3.00	5	3.39	24.96
175	6	10	814.53	2	3.65	0	46.23	1.13	3.71	8	3.99	266.40
175	7	10	923.57	2	4.06	0	49.92	0.73	2.20	9	4.14	232.13
175	9	10	1800.12	5	5.26	0	47.36	0.90	2.38	13	4.32	342.05

Table A.3: Comparison of Global vs. Heuristic approach on SD-CFL instances with $T_{limit} = 1800\,\text{s}$.

A.4 Correlation Analysis of Presolve Indicators

This section reports the detailed Spearman correlation coefficients discussed in Section 7.5. The tables quantify the monotonic relationship between the *Heuristic Performance* metrics:

- $gap^{\%}$: the relative optimality gap (considering only instances that found an optimal solution).
- t_{heur} : the total heuristic runtime.

And the *Presolve Indicators* defined in Chapter 3.3, categorized into:

- **Similarity Indicators:** R_{sim} (Global Similarity Index), $\widehat{R}_{\text{sim}}$ (Minimum Cluster Similarity), and $\overline{R}_{\text{sim}}$ (Maximum Mean Inter-Cluster Similarity).
- **Congestion Indicators:** E_{abs} (Absolute Overload), $R_{\max}$ (Maximum Congestion Ratio), E_{cross} (Cross-Cluster Overload), along with their normalized (E_{norm}) and cluster-level peak variants ($\overline{E}_{\text{abs}}, \overline{R}_{\max}, \overline{E}_{\text{norm}}$).

	Similarity			**Congestion**							
	R_{sim}	$\widehat{R}_{\text{sim}}$	$\overline{R}_{\text{sim}}$	E_{abs}	$R_{\max}$	E_{norm}	$\overline{E}_{\text{abs}}$	$\overline{R}_{\max}$	$\overline{E}_{\text{norm}}$	E_{cross}	R_{cross}
$gap^{\%}$	−0.77	−0.58	0.50	−0.20	−0.12	−0.38	−0.46	−0.10	−0.38	0.69	0.69
t_{heur}	−0.30	0.00	−0.07	0.67	0.41	0.06	0.17	0.27	0.22	0.51	0.26

Table A.4: Spearman correlation coefficients for ST-CPA.

	Similarity			**Congestion**							
	R_{sim}	$\widehat{R}_{\text{sim}}$	$\overline{R}_{\text{sim}}$	E_{abs}	$R_{\max}$	E_{norm}	$\overline{E}_{\text{abs}}$	$\overline{R}_{\max}$	$\overline{E}_{\text{norm}}$	E_{cross}	R_{cross}
$gap^{\%}$	−0.87	−0.59	0.53	−0.16	−0.31	−0.17	−0.29	−0.57	−0.29	0.21	0.21
t_{heur}	−0.40	−0.33	0.02	0.04	0.02	0.01	−0.06	−0.17	−0.06	0.16	0.15

Table A.5: Spearman correlation coefficients for SS-CFLP.

	Similarity			**Congestion**							
	R_{sim}	$\widehat{R}_{\text{sim}}$	$\overline{R}_{\text{sim}}$	E_{abs}	$R_{\max}$	E_{norm}	$\overline{E}_{\text{abs}}$	$\overline{R}_{\max}$	$\overline{E}_{\text{norm}}$	E_{cross}	R_{cross}
$gap^{\%}$	−0.84	−0.65	0.50	−0.09	−0.15	−0.11	−0.33	−0.43	−0.32	0.27	0.26
t_{heur}	−0.41	−0.33	0.19	0.07	0.09	0.02	0.01	−0.03	0.01	0.13	0.11

Table A.6: Spearman correlation coefficients for SD-CFLP.

Bibliography

[1] C. H. Papadimitriou, "On the complexity of integer programming," *J. ACM*, vol. 28, p. 765–768, Oct. 1981.

[2] L. A. Wolsey, *Integer Programming*. New York, NY: Wiley-Interscience, 1998.

[3] R. E. Bixby, "A brief history of linear and mixed-integer programming computation," *Documenta Mathematica*, vol. Extra Volume: Optimization Stories, pp. 107–121, 2012.

[4] T. Achterberg, "Scip: Solving constraint integer programs," *Mathematical Programming Computation*, vol. 1, no. 1, pp. 1–41, 2009.

[5] A. M. Gleixner, G. Hendel, G. Gamrath, T. Achterberg, M. Bastubbe, T. Berthold, P. M. Christophel, K. Jarck, T. Koch, J. T. Linderoth, M. E. Lübbecke, H. D. Mittelmann, D. B. Özyurt, T. K. Ralphs, D. Salvagnin, and Y. Shinano, "MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library," *Mathematical Programming Computation*, vol. 13, no. 3, pp. 443–490, 2021.

[6] M. Skutella, *An Introduction to Network Flows over Time*, pp. 451–482. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009.

[7] S. Irnich and G. Desaulniers, "Shortest path problems with resource constraints," in *Column Generation* (G. Desaulniers, J. Desrosiers, and M. M. Solomon, eds.), Springer, 2005.

[8] L. S. Lasdon, *Optimization Theory for Large Systems*. New York, NY: Macmillan, 1970.

[9] G. B. Dantzig and P. Wolfe, "Decomposition principle for linear programs," *Operations Research*, vol. 8, no. 1, pp. 101–111, 1960.

[10] M. E. Lübbecke and J. Desrosiers, "Selected topics in column generation," *Operations Research*, vol. 53, no. 6, pp. 1007–1023, 2005.

[11] C. Barnhart, E. Johnson, G. L. Nemhauser, M. W. P. Savelsbergh, and P. H. Vance, "Branch-and-price: Column generation for solving huge integer programs," *Operations Research*, vol. 46, no. 3, pp. 316–329, 1998.

[12] G. Cornuéjols, "Valid inequalities for mixed integer linear programs," *Mathematical Programming*, vol. 112, no. 1, pp. 3–44, 2008.

[13] J. P. Vielma, "Mixed integer linear programming formulation techniques," *SIAM Review*, vol. 57, no. 1, pp. 3–57, 2015.

[14] D. Bertsimas and J. Tsitsiklis, *Introduction to Linear Optimization*. 01 1998.

[15] A. Lodi, "Mixed integer programming computation," in *50 Years of Integer Programming 1958–2008* (M. Jünger, T. M. Liebling, D. Naddef, G. L. Nemhauser, W. R. Pulleyblank, G. Reinelt, G. Rinaldi, and L. A. Wolsey, eds.), Springer, 2010.

[16] M. E. Lübbecke, *Column Generation*. John Wiley & Sons, Ltd, 2011.

[17] T. Koch, T. Ralphs, and Y. Shinano, "Could we use a million cores to solve an integer program?," *Mathematical Methods of Operations Research*, vol. 76, no. 1, pp. 67–93, 2012.

[18] G. Desaulniers, J. Desrosiers, and M. M. Solomon, eds., *Column Generation*. Springer, 2005.

[19] J. F. Benders, "Partitioning procedures for solving mixed-variables programming problems," *Numerische Mathematik*, vol. 4, no. 1, pp. 238–252, 1962.

[20] A. M. Geoffrion, "Generalized benders decomposition," *Journal of Optimization Theory and Applications*, vol. 10, no. 4, pp. 237–260, 1972.

[21] R. Rahmaniani, T. G. Crainic, M. Gendreau, and W. Rei, "The benders decomposition algorithm: A literature review," *European Journal of Operational Research*, vol. 259, no. 3, pp. 801–817, 2017.

[22] G. E. Paules and C. A. Floudas, "Apros: Algorithmic development methodology for discrete-continuous optimization problems," *Operations Research*, vol. 37, no. 6, pp. 902–917, 1989.

[23] J. N. Hooker, "Logic-based benders decomposition," *Mathematical Programming*, vol. 96, no. 1, pp. 33–60, 2003.

[24] M. L. Fisher, "The lagrangian relaxation method for solving integer programming problems," *Management science*, vol. 50, no. 12_supplement, pp. 1861–1871, 2004. Originally published in 1981.

[25] P. Wentges, "Weighted dantzig–wolfe decomposition for linear mixed-integer programming," *International Transactions in Operational Research*, vol. 4, no. 2, pp. 151–162, 1997.

[26] V. Maniezzo, M. A. Boschetti, and T. Stützle, *Matheuristics: Algorithms and Implementations*. Springer, 2021.

[27] J. Puchinger and G. R. Raidl, "Combining metaheuristics and exact algorithms in combinatorial optimization: A survey and classification," in *Artificial Intelligence and Knowledge Engineering Applications: A Bioinspired Approach* (J. Mira and J. R. Álvarez, eds.), vol. 3562 of *Lecture Notes in Computer Science*, pp. 41–53, Springer, 2005.

[28] A. K. Jain, M. N. Murty, and P. J. Flynn, "Data clustering: A review," *ACM Computing Surveys*, vol. 31, no. 3, pp. 264–323, 1999.

[29] F. Murtagh and P. Contreras, "Algorithms for hierarchical clustering: An overview," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 2, no. 1, pp. 86–97, 2012.

[30] N. Saitou and M. Nei, "The neighbor-joining method: a new method for reconstructing phylogenetic trees," *Molecular biology and evolution*, vol. 4, no. 4, pp. 406–425, 1987.

[31] I. Elias and J. Lagergren, "Fast neighbor joining," *Theoretical Computer Science*, vol. 410, no. 21–23, pp. 1993–2000, 2009.

[32] B. E. Gillett and L. R. Miller, "A heuristic algorithm for the vehicle-dispatch problem," *Operations Research*, vol. 22, no. 2, pp. 340–349, 1974.

[33] M. L. Fisher and R. Jaikumar, "A generalized assignment heuristic for vehicle routing," *Networks*, vol. 11, no. 2, pp. 109–124, 1981.

[34] R. J. W. Buijs, R. D. van der Mei, E. R. Dugundji, and S. Bhulai, "An effective aggregation heuristic for capacitated facility location problems with many demand points," *Computers & Operations Research*, vol. 183, p. 107153, 2025.

[35] F. Barahona and F. A. Chudak, "Near-optimal solutions to large-scale facility location problems," *Discrete Optimization*, vol. 2, no. 1, pp. 35–50, 2005.

[36] P. Shaw, "Using constraint programming and local search methods to solve vehicle routing problems," in *Principles and Practice of Constraint Programming — CP98* (M. Maher and J.-F. Puget, eds.), vol. 1520 of *Lecture Notes in Computer Science*, pp. 417–431, Springer, 1998.

[37] D. Pisinger and S. Røpke, "A general heuristic for vehicle routing problems," *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.

[38] M. Fischetti and A. Lodi, "Local branching," *Mathematical Programming*, vol. 98, no. 1–3, pp. 23–47, 2003.

[39] E. Danna, E. Rothberg, and C. Le Pape, "Exploring relaxation induced neighborhoods to improve MIP solutions," *Mathematical Programming*, vol. 102, no. 1, pp. 71–90, 2005.

[40] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[41] B. Uçar and C. Aykanat, "Revisiting hypergraph models for sparse matrix partitioning," *SIAM Review*, vol. 49, no. 4, pp. 595–603, 2007.

[42] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.

[43] D. Robinson, E. Feil, and D. Falush, *Bacterial Population Genetics in Infectious Disease*. Wiley, 2010.

[44] J. A. Studier and K. J. Keppler, "A note on the neighbor-joining algorithm of saitou and nei.," *Molecular Biology and Evolution*, vol. 5, pp. 729–731, 11 1988.

[45] M. Simonsen, T. Mailund, and C. N. S. Pedersen, "Rapid neighbour-joining," in *Algorithms in Bioinformatics* (K. A. Crandall and J. Lagergren, eds.), (Berlin, Heidelberg), pp. 113–122, Springer Berlin Heidelberg, 2008.

[46] A. Criscuolo and O. Gascuel, "Fast NJ-like algorithms to deal with incomplete distance matrices," *BMC Bioinformatics*, vol. 9, no. 1, p. 166, 2008.

Ringraziamenti

Un caro ringraziamento va al mio relatore, il Prof. Lorenzo Castelli, non solo per avermi dato la possibilità di svolgere questa tesi, ma per avermi trasmesso l'interesse per questo affascinante argomento durante i suoi insegnamenti, aprendomi nuove prospettive in ambito accademico.

Ringrazio profondamente il mio correlatore, il Dott. Andrea Gasparin, per essere stato un punto di riferimento costante. Grazie per essere stato sempre presente, comprensivo e disponibile.

Un ringraziamento speciale alla mia famiglia per avermi permesso di intraprendere questo percorso di studi e per avermi sempre sostenuto e incoraggiato, credendo in questo cammino e in tutti quelli che verranno.

Ringrazio di cuore Gaia per essere sempre stata al mio fianco e per avermi supportato dal primo all'ultimo giorno, spronandomi a dare il meglio e valorizzando ogni mio piccolo traguardo. Sii fiera di te e credi sempre in quello che fai, con la stessa immensa fiducia che ho io in te.

Ringrazio i colleghi e gli amici conosciuti tra i banchi dell'università, con cui ho condiviso le fatiche dello studio e la gioia dei traguardi raggiunti. Ringrazio i ragazzi di $k7$ per aver trasformato una semplice casa in una seconda famiglia, tra chiacchiere e risate che hanno alleggerito anche le giornate più pesanti. Grazie a chiunque, in questi anni, abbia contribuito a rendere questo percorso indimenticabile.

Infine, un pensiero va anche ai professori che hanno lasciato un segno nel mio percorso di studi. Grazie per aver stimolato la mia curiosità e contribuito a formare la persona che sono oggi, garantendo un clima accademico sereno e collaborativo, ma allo stesso tempo molto stimolante.